

Yantrabyte Solutions - Complete Admin & Database Documentation

Contains Admin Panel code, database schema migrations, and setup steps.

PART 1: OPERATIONAL STEPS & SYSTEM DOCUMENTATION

Step 1: Admin Authentication Flow

The Admin dashboard initializes a Supabase client connection to authenticate users securely. The frontend calls `supabase.auth.getSession()` to restore any persistent user login. If none is found, it renders a login form that validates credentials using `supabase.auth.signInWithPassword()`.

Step 2: Dashboard Overview & Surveillance Metrics

Once authenticated, the dashboard displays counts for tickets, active AMC contracts, and total invoices. It includes a custom live CCTV Surveillance simulator grid displaying workstation layouts and status indicators.

Step 3: Service Ticket Management Lifecycle

Tickets are fetched from the `service_tickets` table. Admins can update status, view details, search, print drop-off receipt PDFs, generate QR tracking labels, and trigger WhatsApp alert updates directly to clients.

Step 4: Invoice Generation & Billing

The Billing module generates custom invoices and uploads them to the Supabase storage bucket, triggering automated receipt emails to client addresses.

Step 5: Supabase Database Schema & Migrations

The database is structured in Supabase Postgres. All tables (`service_tickets`, `invoices`, `customers`, `amc_contracts`, `purchases`, `expenses`, `suppliers`) are built and managed using SQL migrations. RLS (Row Level Security) policies protect admin tables while exposing public forms for anonymous ticket/quote creation.

File: src/admin/AdminPanel.tsx

[illegible]

```
const PAGES_FIELDS: FormField[] = [
  { key: 'title', label: 'Title', type: 'text', required: true },
  { key: 'slug', label: 'Slug', type: 'text', required: true, placeholder: 'e.g. about-us' },
  { key: 'content', label: 'Content (JSON)', type: 'json', rows: 10 },
  { key: 'meta_title', label: 'Meta Title', type: 'text' },
  { key: 'meta_description', label: 'Meta Description', type: 'textarea', rows: 3 },
  { key: 'is_published', label: 'Published', type: 'checkbox' },
  { key: 'page_order', label: 'Page Order', type: 'number' },
];
```

```
const SERVICES_FIELDS: FormField[] = [
  { key: 'title', label: 'Title', type: 'text', required: true },
  { key: 'slug', label: 'Slug', type: 'text', required: true },
  { key: 'short_description', label: 'Short Description', type: 'textarea', rows: 3 },
  { key: 'full_description', label: 'Full Description', type: 'textarea', rows: 6 },
```

```

    { key: 'icon', label: 'Icon', type: 'text', placeholder: 'Camera, Laptop, Monitor, etc.' },
    { key: 'features', label: 'Features (one per line)', type: 'array' },
    { key: 'benefits', label: 'Benefits (one per line)', type: 'array' },
    { key: 'meta_title', label: 'Meta Title', type: 'text' },
    { key: 'meta_description', label: 'Meta Description', type: 'textarea', rows: 3 },
    { key: 'is_featured', label: 'Featured', type: 'checkbox' },
    { key: 'is_published', label: 'Published', type: 'checkbox' },
    { key: 'sort_order', label: 'Sort Order', type: 'number' },
  ];

```

```

const PRODUCTS_FIELDS: FormField[] = [
  { key: 'name', label: 'Product Name', type: 'text', required: true },
  { key: 'slug', label: 'Slug', type: 'text', required: true },
  { key: 'description', label: 'Description', type: 'textarea', rows: 4 },
  { key: 'category', label: 'Category', type: 'text' },
  { key: 'price', label: 'Price', type: 'text' },
  { key: 'image_url', label: 'Image URL', type: 'text' },
  { key: 'features', label: 'Features (one per line)', type: 'array' },
  { key: 'is_featured', label: 'Featured', type: 'checkbox' },
  { key: 'is_published', label: 'Published', type: 'checkbox' },
  { key: 'sort_order', label: 'Sort Order', type: 'number' },
];

```

```

const TESTIMONIALS_FIELDS: FormField[] = [
  { key: 'name', label: 'Client Name', type: 'text', required: true },
  { key: 'company', label: 'Company', type: 'text' },
  { key: 'role', label: 'Role', type: 'text' },
  { key: 'rating', label: 'Rating (1-5)', type: 'number' },
  { key: 'content', label: 'Testimonial Content', type: 'textarea', rows: 5, required: true },
  { key: 'avatar_url', label: 'Avatar URL', type: 'text' },
  { key: 'is_published', label: 'Published', type: 'checkbox' },
  { key: 'sort_order', label: 'Sort Order', type: 'number' },
];

```

```

const BLOG_FIELDS: FormField[] = [
  { key: 'title', label: 'Title', type: 'text', required: true },
  { key: 'slug', label: 'Slug', type: 'text', required: true },
  { key: 'excerpt', label: 'Excerpt', type: 'textarea', rows: 3 },
  { key: 'content', label: 'Content (HTML)', type: 'textarea', rows: 12 },
  { key: 'category', label: 'Category', type: 'text' },
  { key: 'featured_image', label: 'Featured Image URL', type: 'text' },
  { key: 'meta_title', label: 'Meta Title', type: 'text' },
  { key: 'meta_description', label: 'Meta Description', type: 'textarea', rows: 3 },
  { key: 'is_published', label: 'Published', type: 'checkbox' },
  { key: 'sort_order', label: 'Sort Order', type: 'number' },
];

```

```

const TEAM_FIELDS: FormField[] = [
  { key: 'name', label: 'Name', type: 'text', required: true },
  { key: 'role', label: 'Role/Title', type: 'text' },
  { key: 'bio', label: 'Bio', type: 'textarea', rows: 4 },
  { key: 'image_url', label: 'Photo URL', type: 'text' },
  { key: 'is_published', label: 'Published', type: 'checkbox' },
  { key: 'sort_order', label: 'Sort Order', type: 'number' },
];

```

```

const CAREERS_FIELDS: FormField[] = [
  { key: 'title', label: 'Job Title', type: 'text', required: true },
  { key: 'slug', label: 'Slug', type: 'text', required: true },
  { key: 'description', label: 'Description', type: 'textarea', rows: 5 },
  { key: 'requirements', label: 'Requirements (one per line)', type: 'array' },
  { key: 'location', label: 'Location', type: 'text' },
  { key: 'type', label: 'Employment Type', type: 'select', options: [
    { label: 'Full-time', value: 'Full-time' },
    { label: 'Part-time', value: 'Part-time' },
    { label: 'Contract', value: 'Contract' },
    { label: 'Internship', value: 'Internship' },
  ]},
  { key: 'is_active', label: 'Active', type: 'checkbox' },
  { key: 'sort_order', label: 'Sort Order', type: 'number' },
];

```

```

const INDUSTRIES_FIELDS: FormField[] = [
  { key: 'name', label: 'Industry Name', type: 'text', required: true },
  { key: 'slug', label: 'Slug', type: 'text', required: true },
  { key: 'description', label: 'Description', type: 'textarea', rows: 3 },
  { key: 'icon', label: 'Icon', type: 'text', placeholder: 'Building, ShoppingCart, etc.' },
  { key: 'is_published', label: 'Published', type: 'checkbox' },
  { key: 'sort_order', label: 'Sort Order', type: 'number' },
];

```

```

const FAQS_FIELDS: FormField[] = [
  { key: 'question', label: 'Question', type: 'text', required: true },
  { key: 'answer', label: 'Answer', type: 'textarea', rows: 5, required: true },
  { key: 'category', label: 'Category', type: 'text' },
  { key: 'is_published', label: 'Published', type: 'checkbox' },
  { key: 'sort_order', label: 'Sort Order', type: 'number' },
];

```

```

const GALLERY_FIELDS: FormField[] = [
  { key: 'title', label: 'Title', type: 'text', required: true },
  { key: 'before_url', label: 'Before Image URL', type: 'text' },
  { key: 'after_url', label: 'After Image URL', type: 'text' },
  { key: 'category', label: 'Category', type: 'text' },
  { key: 'is_published', label: 'Published', type: 'checkbox' },
  { key: 'sort_order', label: 'Sort Order', type: 'number' },
];

const CLIENT_LOGOS_FIELDS: FormField[] = [
  { key: 'name', label: 'Client Name', type: 'text', required: true },
  { key: 'logo_url', label: 'Logo URL', type: 'text' },
  { key: 'is_published', label: 'Published', type: 'checkbox' },
  { key: 'sort_order', label: 'Sort Order', type: 'number' },
];

const TICKETS_FIELDS: FormField[] = [
  { key: 'ticket_number', label: 'Ticket Number', type: 'text', required: true },
  { key: 'customer_name', label: 'Customer Name', type: 'text', required: true },
  { key: 'customer_email', label: 'Customer Email', type: 'text' },
  { key: 'customer_phone', label: 'Customer Phone', type: 'text', required: true },
  { key: 'customer_address', label: 'Customer Address', type: 'text', rows: 2 },
  { key: 'device_type', label: 'Device/Service Type', type: 'text' },
  { key: 'issue_description', label: 'Issue Description', type: 'text', rows: 4, required: true },
  { key: 'status', label: 'Status', type: 'select', options: [
    { label: 'Open', value: 'open' },
    { label: 'In Progress', value: 'in-progress' },
    { label: 'Completed', value: 'completed' },
    { label: 'Closed', value: 'closed' }
  ]},
  { key: 'priority', label: 'Priority', type: 'select', options: [
    { label: 'Low', value: 'low' },
    { label: 'Medium', value: 'medium' },
    { label: 'High', value: 'high' },
    { label: 'Urgent', value: 'urgent' }
  ]},
  { key: 'notes', label: 'Internal Notes', type: 'text', rows: 3 }
];

const UNIFIED_HEADERS = [
  'Type',
  'No',
  'Date',
  'Customer',
  'Phone',
  'Email',
  'Address',
  'Device / Service',
  'Description',
  'Amount',
  'Payment Status',
  'Status',
  'Assigned To',
  'Link',
];

const serviceTicketRow = (ticket: Partial<ServiceTicket>) => [
  'Service Ticket',
  ticket.ticket_number || '',
  ticket.created_at || new Date().toISOString(),
  ticket.customer_name || '',
  ticket.customer_phone || '',
  ticket.customer_email || '',
  ticket.customer_address || '',
  ticket.device_type || '',
  ticket.issue_description || '',
  '',
  '',
  ticket.status || '',
  ticket.assigned_to || '',
  ticket.ticket_number ? `https://yantrabyte.com/admin` : '',
];

const SECTION_FIELDS: Record<string, FormField[]> = {
  pages: PAGES_FIELDS,
  services: SERVICES_FIELDS,
  products: PRODUCTS_FIELDS,
  testimonials: TESTIMONIALS_FIELDS,
  blog: BLOG_FIELDS,
  team: TEAM_FIELDS,
  careers: CAREERS_FIELDS,
  industries: INDUSTRIES_FIELDS,
  faqs: FAQS_FIELDS,
  gallery: GALLERY_FIELDS,
  'client-logos': CLIENT_LOGOS_FIELDS,
  tickets: TICKETS_FIELDS,
};

// Table Column Configs

```

```

const SECTION_COLUMNS: Record<string, { key: string; label: string }> = {
  pages: [
    { key: 'title', label: 'Title' },
    { key: 'slug', label: 'Slug' },
    { key: 'is_published', label: 'Published' },
    { key: 'page_order', label: 'Order' },
    { key: 'updated_at', label: 'Updated' },
  ],
  services: [
    { key: 'title', label: 'Title' },
    { key: 'slug', label: 'Slug' },
    { key: 'icon', label: 'Icon' },
    { key: 'is_featured', label: 'Featured' },
    { key: 'is_published', label: 'Published' },
    { key: 'sort_order', label: 'Order' },
  ],
  products: [
    { key: 'name', label: 'Name' },
    { key: 'category', label: 'Category' },
    { key: 'price', label: 'Price' },
    { key: 'is_published', label: 'Published' },
    { key: 'sort_order', label: 'Order' },
  ],
  testimonials: [
    { key: 'name', label: 'Name' },
    { key: 'company', label: 'Company' },
    { key: 'rating', label: 'Rating' },
    { key: 'is_published', label: 'Published' },
    { key: 'sort_order', label: 'Order' },
  ],
  blog: [
    { key: 'title', label: 'Title' },
    { key: 'category', label: 'Category' },
    { key: 'is_published', label: 'Published' },
    { key: 'published_at', label: 'Published Date' },
    { key: 'sort_order', label: 'Order' },
  ],
  team: [
    { key: 'name', label: 'Name' },
    { key: 'role', label: 'Role' },
    { key: 'is_published', label: 'Published' },
    { key: 'sort_order', label: 'Order' },
  ],
  careers: [
    { key: 'title', label: 'Title' },
    { key: 'location', label: 'Location' },
    { key: 'type', label: 'Type' },
    { key: 'is_active', label: 'Active' },
    { key: 'sort_order', label: 'Order' },
  ],
  industries: [
    { key: 'name', label: 'Name' },
    { key: 'slug', label: 'Slug' },
    { key: 'is_published', label: 'Published' },
    { key: 'sort_order', label: 'Order' },
  ],
  faqs: [
    { key: 'question', label: 'Question' },
    { key: 'category', label: 'Category' },
    { key: 'is_published', label: 'Published' },
    { key: 'sort_order', label: 'Order' },
  ],
  gallery: [
    { key: 'title', label: 'Title' },
    { key: 'category', label: 'Category' },
    { key: 'is_published', label: 'Published' },
    { key: 'sort_order', label: 'Order' },
  ],
  'client-logos': [
    { key: 'name', label: 'Client Name' },
    { key: 'is_published', label: 'Published' },
    { key: 'sort_order', label: 'Order' },
  ],
  contacts: [
    { key: 'name', label: 'Name' },
    { key: 'email', label: 'Email' },
    { key: 'service', label: 'Service' },
    { key: 'status', label: 'Status' },
    { key: 'created_at', label: 'Date' },
  ],
  tickets: [
    { key: 'ticket_number', label: 'Ticket #' },
    { key: 'customer_name', label: 'Customer' },
    { key: 'customer_phone', label: 'Phone' },
    { key: 'device_type', label: 'Device' },
    { key: 'priority', label: 'Priority' },
    { key: 'status', label: 'Status' },
    { key: 'created_at', label: 'Created' },
  ],
};

```

[illegible]

```

    setLoading(false);
  });
  const { data: { subscription } } = supabase.auth.onAuthStateChange((_evt, session) => {
    setSession(!session);
  });
  return () => subscription.unsubscribe();
}, []);

const handleLogin = async (e: React.FormEvent) => {
  e.preventDefault();
  setAuthError('');
  setLoggingIn(true);
  const { error } = await supabase.auth.signInWithPassword({ email, password });
  if (error) setAuthError(error.message);
  setLoggingIn(false);
};

const handleLogout = async () => {
  await supabase.auth.signOut();
  setSession(false);
};

// --- Toast ---
const showToast = useCallback((message: string, type: 'success' | 'error' = 'success') => {
  setToast({ message, type });
  setTimeout(() => setToast(null), 3000);
}, []);

// --- Print Service Ticket Job Sheet / Drop-off Receipt ---
const printJobSheet = (item: Record<string, unknown>) => {
  const ticketNo = String(item.ticket_number || 'DRAFT');
  const safeText = (value: unknown, fallback = '-') => String(value || fallback)
    .replace(/&/g, '&');
    .replace(/</g, '<');
    .replace(/>/g, '>');
    .replace(/"/g, '"');
    .replace(/'/g, '&#39;');
  const safeMultiline = (value: unknown, fallback: string) => safeText(value, fallback).replace(/\n/g, '<br />');
  const ticketFilename = ticketNo.toLowerCase().endsWith('.pdf') ? ticketNo : `${ticketNo}.pdf`;
  const dateStr = item.created_at ? new Date(String(item.created_at)).toLocaleDateString('en-GB') : new Date().toLocaleDateString('en-GB');

  const element = document.createElement('div');
  element.style.width = '794px';
  element.style.boxSizing = 'border-box';
  element.style.fontFamily = 'Arial, sans-serif';
  element.style.color = '#111111';
  element.style.backgroundColor = 'ffffff';

  element.innerHTML = `
<div style="width: 794px; min-height: 1123px; padding: 20px; box-sizing: border-box; background: #ffffff; position: relative; overflow: hidden;">
  <div style="position: absolute; top: 35%; left: -10%; right: -10%; pointer-events: none; z-index: 1; display: flex; align-items: center; justify-content: center; transform: rotate(180deg);">
    YANTRABYTE SOLUTIONS
  </div>
<div style="border: 2px solid #000000; padding: 14px; min-height: 1083px; box-sizing: border-box; position: relative; z-index: 2;">
  <!-- Header -->
  <table style="width: 100%; border-collapse: collapse; table-layout: fixed; margin-bottom: 16px;">
    <tr>
      <td style="width: 56%; vertical-align: top; padding-right: 16px;">
        
        <div style="font-size: 11px; color: #444444; line-height: 1.45; margin-top: -10px;">
          IT service, repair, and network management experts.<br />
          47A 1st Cross, Sainagar 2nd Stage, Vidyananyapura Post<br />
          Phone: 09986742525 | Email: yantrabyte.solutions@gmail.com
        </div>
      </td>
      <td style="width: 44%; text-align: right; vertical-align: top;">
        <div style="background-color: #0B5394; color: #ffffff; padding: 8px 12px; font-size: 13px; font-weight: bold; text-transform: uppercase; letter-spacing: 0.1em;">
          Service Ticket Receipt
        </div>
        <div style="font-size: 12px; font-weight: bold; margin-top: 10px; color: #111111;">
          Ticket No: <span style="color: #c2410c; font-size: 15px;">${safeText(ticketNo)}</span>
        </div>
        <div style="font-size: 11px; color: #444444; margin-top: 4px;">
          Date: ${dateStr}
        </div>
      </td>
    </tr>
  </table>

  <!-- Divider -->
  <div style="height: 2px; background: #000000; margin-bottom: 16px;"></div>

  <!-- Ticket Details -->
  <table style="width: 100%; border-collapse: collapse; table-layout: fixed; margin-bottom: 12px; font-size: 10px;">
    <colgroup>
      <col style="width: 20%;" />
      <col style="width: 30%;" />
      <col style="width: 18%;" />
    </colgroup>
  </table>
  </div>
  </div>
  `;
};

```

```

        <col style="width: 32%;" />
    </colgroup>
    <tr>
        <td style="border: 1px solid #000000; background: #D9EAF7; padding: 5px 8px; font-weight: bold;">Ticket ID</td>
        <td style="border: 1px solid #000000; padding: 5px 8px; word-break: break-word;">${safeText(ticketNo)}</td>
        <td style="border: 1px solid #000000; background: #D9EAF7; padding: 5px 8px; font-weight: bold;">Date</td>
        <td style="border: 1px solid #000000; padding: 5px 8px; word-break: break-word;">${dateStr}</td>
    </tr>
    <tr>
        <td style="border: 1px solid #000000; background: #D9EAF7; padding: 5px 8px; font-weight: bold;">Customer Name</td>
        <td style="border: 1px solid #000000; padding: 5px 8px; word-break: break-word;">${safeText(item.customer_name)}</td>
        <td style="border: 1px solid #000000; background: #D9EAF7; padding: 5px 8px; font-weight: bold;">Phone</td>
        <td style="border: 1px solid #000000; padding: 5px 8px; word-break: break-word;">${safeText(item.customer_phone)}</td>
    </tr>
    <tr>
        <td style="border: 1px solid #000000; background: #D9EAF7; padding: 5px 8px; font-weight: bold;">Email</td>
        <td colspan="3" style="border: 1px solid #000000; padding: 5px 8px; word-break: break-word;">${safeText(item.customer_email)}</td>
    </tr>
    <tr>
        <td style="border: 1px solid #000000; background: #D9EAF7; padding: 5px 8px; font-weight: bold;">Address</td>
        <td colspan="3" style="border: 1px solid #000000; padding: 5px 8px; line-height: 1.45; word-break: break-word;">${safeText(item.address || item.customer_address)}</td>
    </tr>
    <tr>
        <td style="border: 1px solid #000000; background: #D9EAF7; padding: 5px 8px; font-weight: bold;">Device</td>
        <td style="border: 1px solid #000000; padding: 5px 8px; word-break: break-word;">${safeText(item.device || item.device_type)}</td>
        <td style="border: 1px solid #000000; background: #D9EAF7; padding: 5px 8px; font-weight: bold;">Status</td>
        <td style="border: 1px solid #000000; padding: 5px 8px; text-transform: uppercase; font-weight: bold; color: #0f766e; word-break: break-word;">${safeText(item.status)}</td>
    </tr>
    <tr>
        <td style="border: 1px solid #000000; background: #D9EAF7; padding: 5px 8px; font-weight: bold;">Device Type</td>
        <td style="border: 1px solid #000000; padding: 5px 8px; word-break: break-word;">${safeText(item.device_type)}</td>
        <td style="border: 1px solid #000000; background: #D9EAF7; padding: 5px 8px; font-weight: bold;">Priority</td>
        <td style="border: 1px solid #000000; padding: 5px 8px; text-transform: capitalize; font-weight: bold; color: ${String(item.priority) === 'high' ? 'red' : 'black'};">${item.priority}</td>
    </tr>
</table>

<!-- Customer & Device Details Grid -->
<table style="width: 100%; border-collapse: collapse; table-layout: fixed; margin-bottom: 12px;">
    <tr>
        <!-- Customer info -->
        <td style="width: 50%; border: 1px solid #000000; vertical-align: top;">
            <div style="background-color: #D9EAF7; padding: 6px 10px; font-weight: bold; font-size: 12px; color: #000000; border-bottom: 1px solid #000000;">
                CUSTOMER DETAILS
            </div>
            <div style="padding: 10px; font-size: 11px; line-height: 1.6;">
                <table style="width: 100%; border-collapse: collapse; table-layout: fixed;">
                    <tr><td style="font-weight: bold; width: 34%; vertical-align: top;">Name:</td><td style="word-break: break-word;">${safeText(item.customer_name)}</td></tr>
                    <tr><td style="font-weight: bold; vertical-align: top;">Phone:</td><td style="word-break: break-word;">${safeText(item.customer_phone)}</td></tr>
                    <tr><td style="font-weight: bold; vertical-align: top;">Email:</td><td style="word-break: break-word;">${safeText(item.customer_email)}</td></tr>
                </table>
            </div>
        </td>
        <!-- Device Info -->
        <td style="width: 50%; border: 1px solid #000000; border-left: none; vertical-align: top;">
            <div style="background-color: #D9EAF7; padding: 6px 10px; font-weight: bold; font-size: 12px; color: #000000; border-bottom: 1px solid #000000;">
                DEVICE & SERVICE DETAILS
            </div>
            <div style="padding: 10px; font-size: 11px; line-height: 1.6;">
                <table style="width: 100%; border-collapse: collapse; table-layout: fixed;">
                    <tr><td style="font-weight: bold; width: 38%; vertical-align: top;">Device/Type:</td><td style="word-break: break-word;">${safeText(item.device || item.device_type)}</td></tr>
                    <tr><td style="font-weight: bold; vertical-align: top;">Priority:</td><td style="text-transform: capitalize; font-weight: bold; color: ${String(item.priority) === 'high' ? 'red' : 'black'};">${item.priority}</td></tr>
                    <tr><td style="font-weight: bold; vertical-align: top;">Status:</td><td style="text-transform: uppercase; font-weight: bold; color: #0f766e;">${item.status}</td></tr>
                </table>
            </div>
        </td>
    </tr>
</table>

<!-- Problem Description -->
<div style="border: 1px solid #000000; margin-bottom: 12px;">
    <div style="background-color: #D9EAF7; padding: 5px 10px; font-weight: bold; font-size: 11px; color: #000000; border-bottom: 1px solid #000000;">
        REPORTED CUSTOMER COMPLAINT / ISSUE
    </div>
    <div style="padding: 10px; font-size: 11px; min-height: 50px; line-height: 1.5; color: #111111; word-break: break-word;">
        ${safeMultiline(item.issue_description, 'No complaints specified.')}
    </div>
</div>

<!-- Repair / Drop-off Terms -->
<div style="border: 1px solid #000000; margin-bottom: 10px;">
    <div style="background-color: #D9EAF7; padding: 4px 10px; font-weight: bold; font-size: 11px; color: #000000; border-bottom: 1px solid #000000;">
        TERMS & CONDITIONS
    </div>
    <div style="padding: 8px; font-size: 8px; color: #444444; line-height: 1.5;">
        <ol style="margin: 0; padding-left: 14px;">
            <li><strong>Diagnostic Fee:</strong> Non-refundable diagnostic charge applies regardless of repair approval.</li>
            <li><strong>Data Backup:</strong> Customer is responsible for backing up data. We are not liable for data loss.</li>
            <li><strong>Parts Warranty:</strong> 6-month warranty on replacement parts covering manufacturing defects only.</li>
        </ol>
    </div>
</div>

```



```

        <li><strong>Service Warranty:</strong> 30-day warranty on labor for same-fault recurrence.</li>
        <li><strong>Collection:</strong> Devices uncollected after 30 days incur █50/day storage; disposed after 90 days.</li>
        <li><strong>Limitation:</strong> Our liability shall not exceed the total invoice value.</li>
    </ol>
</div>
</div>

```

```

<!-- Signatures Bottom Section -->

```

```

<div style="margin-top: 10px;">
    <table style="width: 100%; border-collapse: collapse; table-layout: fixed;">
        <tr>
            <!-- Customer Sign -->
            <td style="width: 45%; vertical-align: bottom; padding-right: 20px;">
                <div style="font-size: 10px; font-weight: bold; color: #333333; margin-bottom: 4px;">Customer Acknowledgement</div>
                <div style="width: 200px; border-bottom: 1px solid #555555; margin-bottom: 4px;"></div>
                <div style="font-size: 10px; color: #555555;">Signature</div>
                <div style="width: 200px; border-bottom: 1px solid #555555; margin: 8px 0 4px;"></div>
                <div style="font-size: 10px; color: #555555;">Printed Name</div>
                <div style="width: 200px; border-bottom: 1px solid #555555; margin: 8px 0 4px;"></div>
                <div style="font-size: 10px; color: #555555;">Date</div>
            </td>
            <!-- Yantrabyte Sign + Seal -->
            <td style="width: 55%; text-align: right; vertical-align: bottom; padding-left: 20px;">
                <div style="display: inline-block; text-align: center;">
                    <!-- Round Seal Image with Signature -->
                    <div style="width: 120px; height: 120px; margin: 0 auto 8px;">
                        
                    </div>
                    <div style="width: 200px; border-bottom: 1px solid #555555; margin-bottom: 4px;"></div>
                    <div style="font-size: 11px; font-weight: bold; color: #0B5394;">For Yantrabyte Solutions</div>
                    <div style="font-size: 9px; color: #777777; margin-top: 2px;">Authorized Signatory</div>
                </div>
            </td>
        </tr>
    </table>
</div>
</div>
</div>

```

```

document.body.appendChild(element);
const opt = {
    margin: 0,
    filename: ticketFilename,
    image: { type: 'jpeg' as const, quality: 0.98 },
    html2canvas: { scale: 2, useCORS: true, windowWidth: 794 },
    jsPDF: { unit: 'in' as const, format: 'a4' as const, orientation: 'portrait' as const }
};

```

```

html2pdf().set(opt).from(element).save().then(() => {
    document.body.removeChild(element);
    showToast('Job sheet drop-off receipt downloaded!');
});

```

```

const printLabel = (item: Record<string, unknown>) => {
    const ticketNo = String(item.ticket_number || 'DRAFT');
    const trackUrl = `${window.location.origin}/track?t=${encodeURIComponent(ticketNo)}`;
    const qrUrl = `https://api.qrserver.com/v1/create-qr-code/?size=150x150&data=${encodeURIComponent(trackUrl)}`;

```

```

const printWindow = window.open('', '_blank', 'width=400,height=400');
if (printWindow) {
    printWindow.document.write(`
        <html>
            <head>
                <title>Label - ${ticketNo}</title>
                <style>
                    body { margin: 0; padding: 10px; font-family: Arial, sans-serif; text-align: center; }
                    .label { border: 2px solid #000; padding: 10px; width: 250px; margin: 0 auto; border-radius: 8px; }
                    .title { font-size: 14px; font-weight: bold; margin-bottom: 5px; }
                    .qr { width: 120px; height: 120px; margin: 5px auto; }
                    .ticket { font-size: 18px; font-weight: bold; margin-bottom: 5px; }
                    .info { font-size: 11px; color: #333; overflow: hidden; white-space: nowrap; text-overflow: ellipsis; }
                    @media print {
                        @page { margin: 0; }
                        body { margin: 0.5cm; }
                    }
                </style>
            </head>
            <body>
                <div class="label">
                    <div class="title">YantraByte Solutions</div>
                    <div class="ticket">${ticketNo}</div>
                    
                    <div class="info"><b>Cust:</b> ${item.customer_name || 'N/A'}</div>
                    <div class="info"><b>Device:</b> ${item.device_type || 'N/A'}</div>
                    <div class="info" style="margin-top: 4px; font-size: 10px; font-weight: bold;">Scan to track status</div>
                </div>
            </body>
        `);

```

```

        <script>
            setTimeout(() => { window.print(); window.close(); }, 500);
        </script>
    </body>
</html>
`);
printWindow.document.close();
}
};

const handleInstantBill = (ticket: Record<string, unknown>) => {
    setAutofillTicket(ticket as unknown as ServiceTicket);
    setActiveSection('billing');
    showToast('Switched to billing with pre-filled ticket details!');
};

const sendWhatsAppAlert = (item: Record<string, unknown>) => {
    const name = String(item.customer_name || 'Customer');
    let phone = String(item.customer_phone || '');
    const ticketNo = String(item.ticket_number || 'DRAFT');
    const device = String(item.device_type || 'Device');
    const status = String(item.status || 'open');

    // Format phone number to clean digits (assume Indian +91 if length is 10)
    phone = phone.replace(/\\D/g, '');
    if (phone.length === 10) {
        phone = '91' + phone;
    }

    if (!phone) {
        showToast('No phone number available for WhatsApp', 'error');
        return;
    }

    let text = '';
    if (status === 'open') {
        text = `Hi ${name}, this is Yantrabyte Solutions. We have successfully registered your repair request (Ticket: ${ticketNo}) for your ${device}. Our technician will reach out to you soon.`;
    } else if (status === 'in-progress') {
        text = `Hi ${name}, this is Yantrabyte Solutions. Your ${device} (Ticket: ${ticketNo}) is currently under active diagnostics/repair. We will notify you once the repair is complete.`;
    } else if (status === 'completed') {
        text = `Hi ${name}, great news! Your ${device} (Ticket: ${ticketNo}) has been fully repaired and tested. It is ready for pickup at our workshop. Thank you for choosing Yantrabyte Solutions.`;
    } else if (status === 'closed') {
        text = `Hi ${name}, this is Yantrabyte Solutions. Your repair ticket ${ticketNo} for ${device} has been marked as delivered and closed. Please reach out to us if you need further assistance.`;
    } else {
        text = `Hi ${name}, this is Yantrabyte Solutions. Update regarding your repair ticket ${ticketNo} (${device}). Status: ${status.toUpperCase()}.`;
    }

    const encodedText = encodeURIComponent(text);
    const url = `https://wa.me/${phone}?text=${encodedText}`;
    window.open(url, '_blank');
    showToast('Opening WhatsApp chat...');
};

const sendTicketEmail = async (item: Record<string, unknown>) => {
    const email = String(item.customer_email || '');
    if (!email) {
        showToast('No email address available for this ticket', 'error');
        return;
    }

    showToast('Generating PDF for email...');

    const ticketNo = String(item.ticket_number || '');
    const dateStr = item.created_at ? new Date(String(item.created_at)).toLocaleDateString('en-GB') : new Date().toLocaleDateString('en-GB');
    const safeText = (value: unknown, fallback = '-') => String(value || fallback)
        .replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;').replace(/"/g, '&quot;').replace(/'/g, '&#39;');
    const safeMultiline = (value: unknown, fallback: string) => safeText(value, fallback).replace(/\\n/g, '<br />');

    const element = document.createElement('div');
    element.style.width = '794px';
    element.style.boxSizing = 'border-box';
    element.style.fontFamily = 'Arial, sans-serif';
    element.style.color = '#111111';
    element.style.backgroundColor = '#ffffff';
    element.style.position = 'absolute';
    element.style.left = '-9999px';

    element.innerHTML = `
        <div style="width: 794px; min-height: 1123px; padding: 28px; box-sizing: border-box; background: #ffffff; position: relative; overflow: hidden;">
            <div style="position: absolute; top: 35%; left: -10%; right: -10%; pointer-events: none; z-index: 1; display: flex; align-items: center; justify-content: center; transform: rotate(180deg);">
                YANTRABYTE SOLUTIONS
            </div>
            <div style="border: 2px solid #000000; padding: 18px; min-height: 1067px; box-sizing: border-box; position: relative; z-index: 2;">
                <table style="width: 100%; border-collapse: collapse; table-layout: fixed; margin-bottom: 16px;">
                    <tr>
                        <td style="width: 56%; vertical-align: top; padding-right: 16px;">
                            
                            <div style="font-size: 11px; color: #444444; line-height: 1.45; margin-top: -10px;">

```

```
IT service, repair, and network management experts.<br />
47A 1st Cross, Sainagar 2nd Stage, Vidyananyapura Post<br />
Phone: 09986742525 | Email: yantrabyte.solutions@gmail.com
</div>
</td>
</tr>
<td style="width: 44%; text-align: right; vertical-align: top;">
  <div style="background-color: #0B5394; color: #ffffff; padding: 8px 12px; font-size: 13px; font-weight: bold; text-transform: uppercase; letter-
    <div style="font-size: 12px; font-weight: bold; margin-top: 10px; color: #111111;">Ticket No: <span style="color: #c2410c; font-size: 15px;">${s
    <div style="font-size: 11px; color: #444444; margin-top: 4px;">Date: ${dateStr}</div>
  </td>
</tr>
</table>
<div style="height: 2px; background: #000000; margin-bottom: 16px;"></div>
<table style="width: 100%; border-collapse: collapse; table-layout: fixed; margin-bottom: 18px; font-size: 11px;">
  <colgroup><col style="width: 20%;" /><col style="width: 30%;" /><col style="width: 18%;" /><col style="width: 32%;" /></colgroup>
  <tr>
    <td style="border: 1px solid #000000; background: #D9EAF7; padding: 7px 8px; font-weight: bold;">Ticket ID</td>
    <td style="border: 1px solid #000000; padding: 7px 8px; word-break: break-word;">${safeText(ticketNo)}</td>
    <td style="border: 1px solid #000000; background: #D9EAF7; padding: 7px 8px; font-weight: bold;">Phone</td>
    <td style="border: 1px solid #000000; padding: 7px 8px; word-break: break-word;">${dateStr}</td>
  </tr>
  <tr>
    <td style="border: 1px solid #000000; background: #D9EAF7; padding: 7px 8px; font-weight: bold;">Customer Name</td>
    <td style="border: 1px solid #000000; padding: 7px 8px; word-break: break-word;">${safeText(item.customer_name)}</td>
    <td style="border: 1px solid #000000; background: #D9EAF7; padding: 7px 8px; font-weight: bold;">Date</td>
    <td style="border: 1px solid #000000; padding: 7px 8px; word-break: break-word;">${safeText(item.customer_phone)}</td>
  </tr>
  <tr>
    <td style="border: 1px solid #000000; background: #D9EAF7; padding: 7px 8px; font-weight: bold;">Email</td>
    <td colspan="3" style="border: 1px solid #000000; padding: 7px 8px; word-break: break-word;">${safeText(item.customer_email)}</td>
  </tr>
  <tr>
    <td style="border: 1px solid #000000; background: #D9EAF7; padding: 7px 8px; font-weight: bold;">Address</td>
    <td colspan="3" style="border: 1px solid #000000; padding: 7px 8px; line-height: 1.45; word-break: break-word;">${safeText(item.address || item.cus
  </tr>
  <tr>
    <td style="border: 1px solid #000000; background: #D9EAF7; padding: 7px 8px; font-weight: bold;">Device</td>
    <td style="border: 1px solid #000000; padding: 7px 8px; word-break: break-word;">${safeText(item.device || item.device_type)}</td>
    <td style="border: 1px solid #000000; background: #D9EAF7; padding: 7px 8px; font-weight: bold;">Status</td>
    <td style="border: 1px solid #000000; padding: 7px 8px; text-transform: uppercase; font-weight: bold; color: #0f766e; word-break: break-word;">${s
  </tr>
  <tr>
    <td style="border: 1px solid #000000; background: #D9EAF7; padding: 7px 8px; font-weight: bold;">Device Type</td>
    <td style="border: 1px solid #000000; padding: 7px 8px; word-break: break-word;">${safeText(item.device_type)}</td>
    <td style="border: 1px solid #000000; background: #D9EAF7; padding: 7px 8px; font-weight: bold;">Priority</td>
    <td style="border: 1px solid #000000; padding: 7px 8px; text-transform: capitalize; font-weight: bold; word-break: break-word;">${safeText(item.pr
  </tr>
</table>
<table style="width: 100%; border-collapse: collapse; table-layout: fixed; margin-bottom: 18px;">
  <tr>
    <td style="width: 50%; border: 1px solid #000000; vertical-align: top;">
      <div style="background-color: #D9EAF7; padding: 6px 10px; font-weight: bold; font-size: 12px; color: #000000; border-bottom: 1px solid #000000;">
        <div style="padding: 10px; font-size: 11px; line-height: 1.6;">
          <table style="width: 100%; border-collapse: collapse; table-layout: fixed;">
            <tr><td style="font-weight: bold; width: 34%; vertical-align: top;">Name:</td><td style="word-break: break-word;">${safeText(item.customer_na
            <tr><td style="font-weight: bold; vertical-align: top;">Phone:</td><td style="word-break: break-word;">${safeText(item.customer_phone)}</td>
            <tr><td style="font-weight: bold; vertical-align: top;">Email:</td><td style="word-break: break-word;">${safeText(item.customer_email)}</td>
          </table>
        </div>
      </td>
    <td style="width: 50%; border: 1px solid #000000; border-left: none; vertical-align: top;">
      <div style="background-color: #D9EAF7; padding: 6px 10px; font-weight: bold; font-size: 12px; color: #000000; border-bottom: 1px solid #000000;">REPE
      <div style="padding: 10px; font-size: 11px; line-height: 1.6;">
        <table style="width: 100%; border-collapse: collapse; table-layout: fixed;">
          <tr><td style="font-weight: bold; width: 38%; vertical-align: top;">Device/Type:</td><td style="word-break: break-word;">${safeText(item.dev
          <tr><td style="font-weight: bold; vertical-align: top;">Priority:</td><td style="text-transform: capitalize; font-weight: bold; word-break: b
          <tr><td style="font-weight: bold; vertical-align: top;">Status:</td><td style="text-transform: uppercase; font-weight: bold; color: #0f766e;
        </table>
      </div>
    </td>
  </tr>
</table>
<div style="border: 1px solid #000000; margin-bottom: 18px;">
  <div style="background-color: #D9EAF7; padding: 6px 10px; font-weight: bold; font-size: 12px; color: #000000; border-bottom: 1px solid #000000;">REPE
  <div style="padding: 12px; font-size: 12px; min-height: 78px; line-height: 1.6; color: #111111; word-break: break-word;">${safeMultiline(item.issue_
</div>
<div style="border: 1px solid #000000; margin-bottom: 18px;">
  <div style="background-color: #D9EAF7; padding: 6px 10px; font-weight: bold; font-size: 12px; color: #000000; border-bottom: 1px solid #000000;">DIA
  <div style="padding: 12px; font-size: 12px; min-height: 90px; line-height: 1.6; color: #333333; font-style: italic; word-break: break-word;">${safeM
</div>
<div style="border: 1px solid #000000; margin-bottom: 18px;">
  <div style="background-color: #D9EAF7; padding: 6px 10px; font-weight: bold; font-size: 12px; color: #000000; border-bottom: 1px solid #000000;">
    TERMS & CONDITIONS OF SERVICE
  </div>
  <div style="padding: 10px; font-size: 9px; color: #444444; line-height: 1.55;">
    <ol style="margin: 0; padding-left: 16px;">
      <li><strong>Diagnostic Fee:</strong> A non-refundable diagnostic charge applies to all devices brought in for inspection, regardless of whether r
      <li><strong>Data Backup:</strong> Customer is solely responsible for backing up all data, files, and personal information prior to service. Yantr
    </ol>
  </div>
</div>
```

- Parts Warranty:** All replacement parts carry a standard 6-month warranty from the date of installation unless otherwise specified.
- Service Warranty:** Labor and service work are warranted for 30 days from the date of return. Any recurring issue caused by the device will be replaced at no cost.
- Collection Period:** Devices not collected within 30 days of completion notification will incur a storage fee of \$50/day. Devices collected after 30 days will be sold as-is.
- Estimate Approval:** Repairs will commence only after the customer approves the written estimate. Any additional repairs require a separate estimate and approval.
- E-waste Disposal:** Unclaimed or obsolete electronic components will be recycled or disposed of as per environmental regulations.
- Limitation of Liability:** Yantrabyte Solutions' total liability shall not exceed the total invoice value of the service. We are not responsible for any data loss or damage to devices.

|
 |

Customer Acknowledgement

Signature

Printed Name

Date

 |

For Yantrabyte Solutions

Authorized Signatory

document.body.appendChild(element);

try {

const pdfBlob = await html2pdf()

.set({ margin: 0, filename: `\${ticketNo}.pdf`, image: { type: 'jpeg' as const, quality: 0.98 }, html2canvas: { scale: 2, useCORS: true, windowWidth: 7920 } })

.from(element)

.outputPdf('blob') as Blob;

const reader = new FileReader();

const pdfBase64 = await new Promise<string>((resolve, reject) => {

reader.onloadend = () => {

const result = String(reader.result || '');

resolve(result.includes(',') ? result.split(',')[1] : result);

};

reader.onerror = () => reject(reader.error);

reader.readAsDataURL(pdfBlob);

});

// Upload ticket backup + email via Supabase Edge Function (24/7 online)

const res = await fetch(`\${import.meta.env.VITE\_API\_URL || ''}/api/backups/public-service-ticket`, {

method: 'POST',

headers: { 'Content-Type': 'application/json' },

body: JSON.stringify({

ticket_number: String(item.ticket_number || ''),

customer_name: String(item.customer_name || 'Customer'),

customer_phone: String(item.customer_phone || ''),

customer_email: email,

customer_address: String(item.customer_address || ''),

device_type: String(item.device_type || 'Device'),

issue_description: String(item.issue_description || ''),

priority: String(item.priority || 'medium'),

status: String(item.status || 'open'),

pdfBase64,

pdfFilename: `\${ticketNo}.pdf`,

}),

});

const result = await res.json();

if (result.email?.ok) {

showToast(`Email sent to \${email} with PDF`);

} else {

showToast('Email skipped (no Gmail configured or invalid email)');

}

} catch (err) {

const errorMsg = err instanceof Error ? err.message : String(err);

showToast('Failed to send email: ' + errorMsg, 'error');

} finally {

document.body.removeChild(element);

}

};

// --- Data Fetching ---

```

const fetchData = useCallback(async (section: Section) => {
  const config = SECTION_CONFIG[section];
  if (!config.table) return;
  setDataLoading(true);
  const { data: rows, error } = await supabase
    .from(config.table as string)
    .select('*')
    .order(config.orderField as string, { ascending: true });
  if (error) {
    showToast('Error loading data: ' + error.message, 'error');
  } else {
    setData(prev => ({ ...prev, [section]: rows || [] }));
  }
  setDataLoading(false);
}, [showToast]);

const [isSyncing, setIsSyncing] = useState(false);
const [isExportingTickets, setIsExportingTickets] = useState(false);

const exportServiceTicketsExcel = async () => {
  setIsExportingTickets(true);
  try {
    const { data: ticketsData, error } = await supabase
      .from('service_tickets')
      .select('*')
      .order('created_at', { ascending: false });

    if (error) throw error;

    const tickets = (ticketsData || []) as ServiceTicket[];
    downloadExcelWorkbook(`service-tickets-${new Date().toISOString().slice(0, 10)}.xls`, [
      {
        name: 'Service Tickets',
        rows: [
          UNIFIED_HEADERS,
          ...tickets.map(serviceTicketRow),
        ],
      },
    ],);

    showToast('Service tickets exported to Excel!');
  } catch (err) {
    const errorMsg = err instanceof Error ? err.message : String(err);
    showToast(errorMsg || 'Failed to export service tickets', 'error');
  } finally {
    setIsExportingTickets(false);
  }
};

const TICKET_SHEET_NAME = 'Service Tickets';
const TICKET_SHEET_HEADERS = ['Ticket No', 'Date', 'Customer', 'Phone', 'Email', 'Address', 'Device', 'Issue', 'Priority', 'Status', 'Notes', 'PDF Link'];

const backupTicketToGoogleSheet = async (ticket: Partial<ServiceTicket>) => {
  try {
    const result = await appendBackupRow({
      sheetName: TICKET_SHEET_NAME,
      headers: TICKET_SHEET_HEADERS,
      row: [
        ticket.ticket_number || '',
        ticket.created_at ? new Date(ticket.created_at).toLocaleDateString('en-GB') : '',
        ticket.customer_name || '',
        ticket.customer_phone || '',
        ticket.customer_email || '',
        ticket.customer_address || '',
        ticket.device_type || '',
        ticket.issue_description || '',
        ticket.priority || '',
        ticket.status || '',
        ticket.notes || '',
        '',
      ],
    });
    if (result.ok) {
      showToast('Sheet backup updated');
    } else if (!result.skipped) {
      showToast('Sheet backup failed: ' + (result.error || 'unknown'), 'error');
    }
  } catch {
    showToast('Sheet backup unavailable (server down?)', 'error');
  }
};

const backupTicketToDrive = async (ticketId: string) => {
  try {
    // Retrieve ticket data and fire edge function notification
    const { data: ticketData } = await supabase
      .from('service_tickets')
      .select('*')

```

```

    .eq('id', ticketId)
    .single();
  if (!ticketData) return;
  // Fire Telegram notification for admin backup
  supabase.functions.invoke('notify-telegram', {
    body: {
      message:
        `■ <b>Ticket Backup (Admin)</b>\n` +
        `Ticket: ${ticketData.ticket_number || 'N/A'}\n` +
        `Customer: ${ticketData.customer_name || 'N/A'}\n` +
        `Phone: ${ticketData.customer_phone || 'N/A'}\n` +
        `Device: ${ticketData.device_type || 'N/A'}\n` +
        `Issue: ${String(ticketData.issue_description || '').slice(0, 100)}\n`,
    },
  }).catch(() => {});
} catch {
  // Non-fatal
}
};

const syncFromGoogleSheet = async () => {
  setIsSyncing(true);
  try {
    // Fetching the Google Sheet CSV with multi-stage robust fallbacks (direct first, then proxies)
    const csvUrl = 'https://docs.google.com/spreadsheets/d/1y6dyRVn0seq5qZfVmThTXJHiEoyG9kgoLe0j9WZbB0c/export?format=csv&gid=1073064749';
    let response;
    let csvText = '';

    try {
      // 1. Try Direct Fetch first (Google Sheets exports are public and support CORS directly in modern browsers)
      response = await fetch(csvUrl);
      if (!response.ok) throw new Error('Direct fetch returned status ' + response.status);
      csvText = await response.text();
    } catch (directErr) {
      console.warn('Direct Google Sheet fetch failed, trying corsproxy.io proxy...', directErr);
      try {
        // 2. Try corsproxy.io as the second stage (super fast and highly reliable proxy)
        response = await fetch('https://corsproxy.io/?${encodeURIComponent(csvUrl)}');
        if (!response.ok) throw new Error('corsproxy.io returned status ' + response.status);
        csvText = await response.text();
      } catch (proxy1Err) {
        console.warn('corsproxy.io proxy failed, trying api.allorigins.win proxy...', proxy1Err);
        // 3. Try allorigins as the third stage fallback
        const targetUrl = encodeURIComponent(csvUrl);
        response = await fetch('https://api.allorigins.win/raw?url=${targetUrl}');
        if (!response.ok) throw new Error('allorigins proxy returned status ' + response.status);
        csvText = await response.text();
      }
    }
  }
}

const lines = csvText.split('\n');
const newTickets = [];
let inCsv = false;

for (let i = 0; i < lines.length; i++) {
  const line = lines[i].trim();
  if (line.match(/^\d{1,2}\.\d{1,2}\.\d{4}/)) {
    inCsv = true;
  }
  if (inCsv && line && !line.includes(',,,,,,,,,,,,,,,,,,,,,,,,,,,,,')) {
    const regex = /(,(?=(?:[^\"]{2})*[^\"]*$))/;
    const columns = line.split(regex);
    if (columns.length > 9) {
      const name = columns[1]?.replace(/'/g, '"').replace(/\"|\"$/g, '') || '';
      const phone = columns[2]?.replace(/'/g, '"').replace(/\"|\"$/g, '') || '';
      const email = columns[3]?.replace(/'/g, '"').replace(/\"|\"$/g, '') || '';
      const device = columns[5]?.replace(/'/g, '"').replace(/\"|\"$/g, '') || '';
      const issue = columns[6]?.replace(/'/g, '"').replace(/\"|\"$/g, '') || '';
      let ticketId = columns[20]?.replace(/'/g, '"').replace(/\"|\"$/g, '');
      const statusRaw = columns[12]?.replace(/'/g, '"').replace(/\"|\"$/g, '').toLowerCase() || 'open';

      let status = 'open';
      if (statusRaw.includes('progress')) status = 'in-progress';
      if (statusRaw.includes('closed') || statusRaw.includes('resolved')) status = 'closed';
      if (!ticketId) ticketId = `YBS-FORM-${Date.now()}-${i}`;

      if (name) {
        newTickets.push({
          ticket_number: ticketId,
          customer_name: name,
          customer_phone: phone,
          customer_email: email,
          device_type: device,
          issue_description: issue,
          status: status
        });
      }
    }
  }
}
}

```

```

    }
  }

  if (newTickets.length > 0) {
    const { error } = await supabase.from('service_tickets').upsert(newTickets, { onConflict: 'ticket_number', ignoreDuplicates: true });
    if (error) throw error;
    showToast('Successfully synced from Google Form!');
    fetchData('tickets');
  } else {
    showToast('No valid tickets found in sheet');
  }
} catch (err) {
  const errorMsg = err instanceof Error ? err.message : String(err);
  showToast('Error syncing: ' + errorMsg, 'error');
}
setIsSyncing(false);
};

useEffect(() => {
  if (session && activeSection !== 'dashboard') {
    fetchData(activeSection);
  }
}, [session, activeSection, fetchData]);

// --- Realtime Auto-Refresh ---
useEffect(() => {
  if (!session) return;

  const channel = supabase
    .channel('admin-panel-realtime')
    .on('postgres_changes', { event: '*', schema: 'public', table: 'service_tickets' }, () => {
      if (activeSection === 'tickets' || activeSection === 'dashboard') {
        fetchData(activeSection);
      }
    })
    .on('postgres_changes', { event: '*', schema: 'public', table: 'invoices' }, () => {
      if (activeSection === 'dashboard') {
        fetchData(activeSection);
      }
    })
    .on('postgres_changes', { event: '*', schema: 'public', table: 'purchases' }, () => {
      if (activeSection === 'dashboard') {
        fetchData(activeSection);
      }
    })
    .subscribe();

  return () => {
    supabase.removeChannel(channel);
  };
}, [session, activeSection, fetchData]);

// --- Dashboard Stats ---
useEffect(() => {
  if (session && activeSection === 'dashboard') {
    setDashboardLoading(true);

    Promise.all([
      supabase.from('invoices').select('*').order('date', { ascending: false }),
      supabase.from('service_tickets').select('*').order('created_at', { ascending: false })
    ]).then(([invoicesRes, ticketsRes]) => {
      if (invoicesRes.data) {
        setFinancialInvoices(invoicesRes.data as Invoice[]);
      }
      if (ticketsRes.data) {
        setFinancialTickets(ticketsRes.data as ServiceTicket[]);
      }
      setDashboardLoading(false);
    }).catch(err => {
      console.error('Error loading dashboard financials:', err);
      setDashboardLoading(false);
    });
  }
}, [session, activeSection]);

// --- Fetch customers when tickets section is active ---
useEffect(() => {
  if (session && activeSection === 'tickets') {
    supabase.from('customers').select('*').order('name').then(({ data, error }) => {
      if (error) {
        console.error('Error fetching customers:', error);
        return;
      }
      if (data) setCustomersList(data as Customer[]);
    });
  }
}, [session, activeSection]);

// --- CRUD ---

```

```

const openAddForm = () => {
  const fields = SECTION_FIELDS[activeSection];
  if (!fields) return;
  setEditingItem(null);
  const defaultData = getDefaultForm(fields);

  if (activeSection === 'tickets') {
    const tickets = (data.tickets || []) as ServiceTicket[];
    const now = new Date();
    const currentYear = now.getFullYear();
    const currentMonth = now.getMonth();
    const startYear = currentMonth < 3 ? currentYear - 1 : currentYear;
    const datePrefix = `${startYear}-${startYear + 1}`;
    const prefixString = `YBS-${datePrefix}-`;

    let maxSeq = 0;
    tickets.forEach((t) => {
      if (t.ticket_number?.startsWith(prefixString)) {
        const match = String(t.ticket_number).match(/-(\d+)$/);
        if (match) {
          const num = parseInt(match[1], 10);
          if (!isNaN(num) && num > maxSeq) maxSeq = num;
        }
      }
    });
    defaultData['ticket_number'] = `${prefixString}${String(maxSeq + 1).padStart(3, '0')}`;
    defaultData['status'] = 'open';
    defaultData['priority'] = 'medium';
  }

  setFormData(defaultData);
  setFormError('');
  setShowForm(true);
};

const openEditForm = (item: Record<string, unknown>) => {
  const fields = SECTION_FIELDS[activeSection];
  if (!fields) return;
  setEditingItem(item);
  setFormData(recordToForm(item, fields));
  setFormError('');
  setShowForm(true);
};

const handleSave = async () => {
  const config = SECTION_CONFIG[activeSection];
  const fields = SECTION_FIELDS[activeSection];
  if (!config.table || !fields) return;

  setFormSaving(true);
  setFormError('');

  const record = formToRecord(formData, fields);

  try {
    if (editingItem) {
      const { error } = await supabase
        .from(config.table as string)
        .update(record)
        .eq('id', editingItem.id);
      if (error) throw error;
      if (activeSection === 'tickets') {
        backupTicketToGoogleSheet({ ...(editingItem as unknown as Partial<ServiceTicket>), ...(record as Partial<ServiceTicket>) });
        void backupTicketToDrive(String(editingItem.id));
      }
      showToast('Item updated successfully');
    } else {
      const { data: inserted, error } = await supabase
        .from(config.table as string)
        .insert([record])
        .select('id');
      if (error) throw error;
      if (activeSection === 'tickets' && inserted && inserted.length > 0) {
        backupTicketToGoogleSheet(record as Partial<ServiceTicket>);
        void backupTicketToDrive(String(inserted[0].id));
      }
      showToast('Item created successfully');
    }
    setShowForm(false);
    setEditingItem(null);
    fetchData(activeSection);
  } catch (err: unknown) {
    const msg = err instanceof Error ? err.message : (err as { message?: string })?.message || 'Unknown error';
    setFormError(msg);
  }
  setFormSaving(false);
};

const handleDelete = async (id: string) => {

```



```

const config = SECTION_CONFIG[activeSection];
if (!config.table) return;
const { error } = await supabase
  .from(config.table as string)
  .delete()
  .eq('id', id);
if (error) {
  showToast('Error deleting: ' + error.message, 'error');
} else {
  showToast('Item deleted successfully');
  fetchData(activeSection);
}
setDeleteConfirm(null);
};

const togglePublished = async (item: Record<string, unknown>) => {
  const config = SECTION_CONFIG[activeSection];
  if (!config.table || !config.publishedField) return;
  const newVal = !item[config.publishedField];
  const { error } = await supabase
    .from(config.table as string)
    .update({ [config.publishedField]: newVal })
    .eq('id', item.id);
  if (error) {
    showToast('Error updating: ' + error.message, 'error');
  } else {
    showToast(newVal ? 'Item published' : 'Item unpublished');
    fetchData(activeSection);
  }
};

// --- Contact Status ---
const updateContactStatus = async (id: string, status: string) => {
  const { error } = await supabase
    .from('contact_submissions')
    .update({ status })
    .eq('id', id);
  if (error) {
    showToast('Error updating status: ' + error.message, 'error');
  } else {
    showToast('Status updated');
    fetchData('contacts');
  }
};

const updateTicketStatus = async (id: string, status: string) => {
  const { error } = await supabase
    .from('service_tickets')
    .update({ status })
    .eq('id', id);
  if (error) {
    showToast('Error updating status: ' + error.message, 'error');
  } else {
    showToast('Status updated to ${status}`');
    fetchData('tickets');
  }
};

// --- Site Settings ---
const [settingsRows, setSettingsRows] = useState<SiteSetting[]>([]);
const [settingsLoading, setSettingsLoading] = useState(false);
const [settingsForm, setSettingsForm] = useState<Record<string, string>>({});
const [newSettingKey, setNewSettingKey] = useState('');
const [newSettingValue, setNewSettingValue] = useState('');

useEffect(() => {
  if (session && activeSection === 'settings') {
    setSettingsLoading(true);
    supabase.from('site_settings').select('*').order('key').then(({ data, error }) => {
      if (error) showToast('Error loading settings: ' + error.message, 'error');
      else {
        const rows = (data as SiteSetting[]) || [];
        setSettingsRows(rows);
        const form: Record<string, string> = {};
        rows.forEach(s => { form[s.key] = s.value; });
        setSettingsForm(form);
      }
      setSettingsLoading(false);
    });
  }
}, [session, activeSection, showToast]);

const saveSettings = async () => {
  setFormSaving(true);
  try {
    for (const row of settingsRows) {
      const newVal = settingsForm[row.key];
      if (newVal !== undefined && newVal !== row.value) {

```

```

        const { error } = await supabase.from('site_settings').update({ value: newVal }).eq('id', row.id);
        if (error) throw error;
    }
}
showToast('Settings saved successfully');
} catch (err: unknown) {
    const msg = err instanceof Error ? err.message : 'Error saving settings';
    showToast(msg, 'error');
}
setFormSaving(false);
};

const addSetting = async () => {
    if (!newSettingKey.trim()) return;
    const { error } = await supabase.from('site_settings').insert([{ key: newSettingKey, value: newSettingValue }]);
    if (error) { showToast('Error: ' + error.message, 'error'); return; }
    setNewSettingKey('');
    setNewSettingValue('');
    showToast('Setting added');
    // Reload
    const { data } = await supabase.from('site_settings').select('*').order('key');
    const rows = (data as SiteSetting[]) || [];
    setSettingsRows(rows);
    const form: Record<string, string> = {};
    rows.forEach(s => { form[s.key] = s.value; });
    setSettingsForm(form);
};

const deleteSetting = async (id: string) => {
    const { error } = await supabase.from('site_settings').delete().eq('id', id);
    if (error) { showToast('Error: ' + error.message, 'error'); return; }
    showToast('Setting deleted');
    const { data } = await supabase.from('site_settings').select('*').order('key');
    const rows = (data as SiteSetting[]) || [];
    setSettingsRows(rows);
    const form: Record<string, string> = {};
    rows.forEach(s => { form[s.key] = s.value; });
    setSettingsForm(form);
};

// --- Filtered data ---
const getFilteredData = () => {
    const sectionData = (data[activeSection] || []) as Record<string, unknown>[];
    if (!searchQuery.trim()) return sectionData;
    const q = searchQuery.toLowerCase();
    return sectionData.filter(item =>
        Object.values(item).some(v => String(v).toLowerCase().includes(q))
    );
};

// --- Loading screen ---
if (loading) {
    return (
        <div className="min-h-screen bg-[#0B1120] flex items-center justify-center">
            <Loader2 className="w-8 h-8 text-[#0EA5E9] animate-spin" />
        </div>
    );
}

// --- Login Screen ---
if (!session) {
    return (
        <div className="min-h-screen bg-[#0B1120] flex items-center justify-center px-4">
            <div className="w-full max-w-md">
                <div className="text-center mb-8">
                    <div className="w-16 h-16 mx-auto rounded-2xl bg-[#0EA5E9]/10 flex items-center justify-center mb-4">
                        <LayoutDashboard className="w-8 h-8 text-[#0EA5E9]" />
                    </div>
                    <h1 className="text-2xl font-bold text-white">Admin Panel</h1>
                    <p className="text-[#94A3B8] text-sm mt-2">Yantrabyte Solutions</p>
                </div>
                <form onSubmit={handleLogin} className="backdrop-blur-xl bg-white/5 border border-white/10 rounded-2xl p-8 space-y-5">
                    {authError && (
                        <div className="flex items-center gap-2 text-red-400 text-sm bg-red-500/10 border border-red-500/20 rounded-xl p-3">
                            <AlertCircle className="w-4 h-4 shrink-0" />
                            {authError}
                        </div>
                    )}
                    <div>
                        <label className="block text-white text-sm font-medium mb-2">Email</label>
                        <input
                            type="email"
                            value={email}
                            onChange={e => setEmail(e.target.value)}
                            required
                            placeholder="admin@yantrabyte.com"
                            className="w-full px-4 py-3 rounded-xl bg-white/5 border border-white/10 text-white placeholder-[#64748B] text-sm focus:outline-none focus:bor
                        />
                    </div>
                </form>
            </div>
        </div>
    );
}

```

```

    </div>
    <div>
      <label className="block text-white text-sm font-medium mb-2">Password</label>
      <input
        type="password"
        value={password}
        onChange={e => setPassword(e.target.value)}
        required
        placeholder="Enter your password"
        className="w-full px-4 py-3 rounded-xl bg-white/5 border border-white/10 text-white placeholder-[#64748B] text-sm focus:outline-none focus:bor
      />
    </div>
    <button
      type="submit"
      disabled={loggingIn}
      className="w-full py-3 rounded-xl bg-[#0EA5E9] hover:bg-[#0284C7] text-white font-semibold transition-all disabled:opacity-50 flex items-center
    >
      {loggingIn ? <Loader2 className="w-5 h-5 animate-spin" /> : null}
      {loggingIn ? 'Signing in...' : 'Sign In'}
    </button>
  </form>
</div>
</div>
);
}

// --- Sidebar ---
const sidebarItems: { section: Section; label: string; icon: React.ElementType }[] = [
  { section: 'dashboard', label: 'Dashboard', icon: LayoutDashboard },
  { section: 'pages', label: 'Pages', icon: FileText },
  { section: 'services', label: 'Services', icon: Wrench },
  { section: 'products', label: 'Products', icon: Package },
  { section: 'testimonials', label: 'Testimonials', icon: MessageSquareQuote },
  { section: 'blog', label: 'Blog Posts', icon: PenTool },
  { section: 'team', label: 'Team Members', icon: Users },
  { section: 'careers', label: 'Careers', icon: Briefcase },
  { section: 'industries', label: 'Industries', icon: Building2 },
  { section: 'faqs', label: 'FAQs', icon: HelpCircle },
  { section: 'gallery', label: 'Gallery', icon: Image },
  { section: 'client-logos', label: 'Client Logos', icon: Award },
  { section: 'contacts', label: 'Contact Submissions', icon: Mail },
  { section: 'tickets', label: 'Service Ticket', icon: Ticket },
  { section: 'external-repairs', label: 'External Repairs', icon: ArrowLeftRight },
  { section: 'billing', label: 'Billing Software', icon: Receipt },
  { section: 'purchase', label: 'Purchase Entry', icon: Truck },
  { section: 'settings', label: 'Site Settings', icon: Settings },
];

const renderCellValue = (item: Record<string, unknown>, colKey: string) => {
  const val = item[colKey];
  if (colKey === 'is_published' || colKey === 'is_active' || colKey === 'is_featured') {
    return val ? (
      <span className="inline-flex items-center gap-1 px-2 py-0.5 rounded-full text-xs font-medium bg-emerald-500/10 text-emerald-400 border border-emerald-500/10">
        <CheckCircle className="w-3 h-3" /> Yes
      </span>
    ) : (
      <span className="inline-flex items-center gap-1 px-2 py-0.5 rounded-full text-xs font-medium bg-white/5 text-[#64748B] border border-white/10">
        <X className="w-3 h-3" /> No
      </span>
    );
  }
}
if (colKey === 'status') {
  const status = String(val);
  if (activeSection === 'tickets') {
    const colors: Record<string, string> = {
      open: 'bg-blue-500/10 text-blue-400 border-blue-500/20 focus:border-blue-500/50',
      'in-progress': 'bg-yellow-500/10 text-yellow-400 border-yellow-500/20 focus:border-yellow-500/50',
      completed: 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20 focus:border-emerald-500/50',
      closed: 'bg-gray-500/10 text-gray-400 border-gray-500/20 focus:border-gray-500/50',
    };
    return (
      <select
        value={status}
        onChange={e => updateTicketStatus(String(item.id), e.target.value)}
        className={`px-2 py-0.5 rounded-full text-xs font-medium border bg-[#0F172A] text-white cursor-pointer outline-none transition-all ${colors[status]}
      >
        <option value="open">Open</option>
        <option value="in-progress">In Progress</option>
        <option value="completed">Completed</option>
        <option value="closed">Closed</option>
      </select>
    );
  }
}
const colors: Record<string, string> = {
  new: 'bg-blue-500/10 text-blue-400 border-blue-500/20',
  read: 'bg-yellow-500/10 text-yellow-400 border-yellow-500/20',
  replied: 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20',
  open: 'bg-blue-500/10 text-blue-400 border-blue-500/20',

```

```

    'in-progress': 'bg-yellow-500/10 text-yellow-400 border-yellow-500/20',
    completed: 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20',
    closed: 'bg-gray-500/10 text-gray-400 border-gray-500/20',
  };
  return (
    <span className={`inline-flex items-center px-2 py-0.5 rounded-full text-xs font-medium border ${colors[status]} || 'bg-white/5 text-[#64748B] border-w
      {status}
    </span>
  );
}
if (colKey === 'created_at' || colKey === 'updated_at' || colKey === 'published_at') {
  return <span className="text-[#94A3B8] text-xs">{formatDate(String(val))}</span>;
}
if (colKey === 'rating') {
  return <span className="text-yellow-400">{val as React.ReactNode}</span>;
}
if (typeof val === 'object' && val !== null) {
  return <span className="text-[#64748B] text-xs">{truncateStr(JSON.stringify(val), 30)}</span>;
}
return <span className="text-white text-sm">{truncateStr(String(val || '-'), 40)}</span>;
};

// --- Render Dashboard ---
const renderDashboard = () => {
  if (dashboardLoading) {
    return (
      <div className="flex flex-col items-center justify-center py-20 text-white">
        <Loader2 className="w-10 h-10 text-[#0EA5E9] animate-spin mb-4" />
        <p className="text-sm text-[#94A3B8]">Assembling Financial Ledger & Operational Metrics...</p>
      </div>
    );
  }
}

const totalInvoiced = financialInvoices
  .filter(inv => inv.doc_type === 'Invoice')
  .reduce((sum, inv) => sum + (inv.grand_total || 0), 0);

const outstandingDues = financialInvoices
  .filter(inv => inv.doc_type === 'Invoice')
  .reduce((sum, inv) => sum + (inv.balance_due || 0), 0);

const collectedAmount = totalInvoiced - outstandingDues;

// Service Ticket Stats
const openTickets = financialTickets.filter(t => t.status === 'open').length;
const inProgressTickets = financialTickets.filter(t => t.status === 'in-progress').length;
const completedTickets = financialTickets.filter(t => t.status === 'completed').length;
const closedTickets = financialTickets.filter(t => t.status === 'closed').length;
const pendingTickets = openTickets + inProgressTickets;

interface OutstandingClient {
  customer_name: string;
  customer_phone: string;
  balance_due: number;
  invoices: string[];
}

// Get list of outstanding clients
const outstandingClients = financialInvoices
  .filter(inv => inv.doc_type === 'Invoice' && (inv.balance_due || 0) > 0)
  .reduce((acc: OutstandingClient[], inv) => {
    const name = String(inv.customer_name || 'Customer');
    const phone = String(inv.phone || '');
    const due = inv.balance_due || 0;

    const existing = acc.find(c => c.customer_name?.toLowerCase() === name.toLowerCase());
    if (existing) {
      existing.balance_due += due;
      if (!existing.invoices.includes(inv.invoice_no)) {
        existing.invoices.push(inv.invoice_no);
      }
    } else {
      acc.push({
        customer_name: name,
        customer_phone: phone,
        balance_due: due,
        invoices: [inv.invoice_no]
      });
    }
  }, []);
return acc;
}, [])
.sort((a, b) => b.balance_due - a.balance_due);

// Formats and opens direct WhatsApp reminder
const sendDuesReminder = (client: OutstandingClient) => {
  const name = client.customer_name;
  let phone = client.customer_phone.replace(/\D/g, '');
  if (phone.length === 10) {

```

```

    phone = '91' + phone;
}
if (!phone) {
    showToast('No phone number available for client', 'error');
    return;
}

const text = `Hi ${name}, this is Ramesh from YantraByte Solutions. A friendly reminder that your outstanding balance of ₹${client.balance_due.toLocaleString('en-IN')} is due. Please make the payment at the earliest to avoid any inconvenience. Thank you for your business.`;
const encoded = encodeURIComponent(text);
window.open(`https://wa.me/${phone}?text=${encoded}`, '_blank');
showToast('Opening WhatsApp reminder chat...');
};

return (
    <div className="space-y-6">
        <div>
            <h2 className="text-2xl font-bold text-white">YantraByte Solutions • Enterprise Ledger</h2>
            <p className="text-xs text-[#94A3B8] mt-1">Real-time financial collection ledger and workshop diagnostics tracking</p>
        </div>

        /* --- PREMIUM FINANCIAL KPI CARDS --- */
        <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4">
            <div className="backdrop-blur-xl bg-white/5 border border-white/10 rounded-2xl p-5 hover:border-indigo-500/30 transition-all duration-300">
                <div className="flex items-center justify-between mb-3">
                    <div className="w-10 h-10 rounded-xl bg-indigo-500/10 flex items-center justify-center">
                        <DollarSign className="w-5 h-5 text-indigo-400" />
                    </div>
                    <span className="text-xs font-semibold text-[#94A3B8] uppercase">Total Invoiced</span>
                </div>
                <div className="text-2xl font-bold text-white font-mono">₹{totalInvoiced.toLocaleString('en-IN')}</div>
                <p className="text-[10px] text-[#64748B] mt-1">Sum of lifetime generated invoices</p>
            </div>

            <div className="backdrop-blur-xl bg-white/5 border border-white/10 rounded-2xl p-5 hover:border-emerald-500/30 transition-all duration-300">
                <div className="flex items-center justify-between mb-3">
                    <div className="w-10 h-10 rounded-xl bg-emerald-500/10 flex items-center justify-center">
                        <TrendingUp className="w-5 h-5 text-emerald-400" />
                    </div>
                    <span className="text-xs font-semibold text-[#94A3B8] uppercase">Realized Revenue</span>
                </div>
                <div className="text-2xl font-bold text-emerald-400 font-mono">₹{collectedAmount.toLocaleString('en-IN')}</div>
                <p className="text-[10px] text-[#64748B] mt-1">Total cash successfully collected</p>
            </div>

            <div className="backdrop-blur-xl bg-white/5 border border-white/10 rounded-2xl p-5 hover:border-rose-500/30 transition-all duration-300">
                <div className="flex items-center justify-between mb-3">
                    <div className="w-10 h-10 rounded-xl bg-rose-500/10 flex items-center justify-center">
                        <Clock className="w-5 h-5 text-rose-400" />
                    </div>
                    <span className="text-xs font-semibold text-[#94A3B8] uppercase">Outstanding Balance</span>
                </div>
                <div className="text-2xl font-bold text-rose-400 font-mono font-bold">₹{outstandingDues.toLocaleString('en-IN')}</div>
                <p className="text-[10px] text-[#64748B] mt-1">Pending payments collection backlog</p>
            </div>

            <div className="backdrop-blur-xl bg-white/5 border border-white/10 rounded-2xl p-5 hover:border-amber-500/30 transition-all duration-300">
                <div className="flex items-center justify-between mb-3">
                    <div className="w-10 h-10 rounded-xl bg-amber-500/10 flex items-center justify-center">
                        <Activity className="w-5 h-5 text-amber-400" />
                    </div>
                    <span className="text-xs font-semibold text-[#94A3B8] uppercase">Active Service Tickets</span>
                </div>
                <div className="text-2xl font-bold text-amber-400 font-mono">{pendingTickets}</div>
                <p className="text-[10px] text-[#64748B] mt-1">{openTickets} open • {inProgressTickets} in-progress</p>
            </div>
        </div>

        /* --- SECOND LEVEL DETAILS GRID --- */
        <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
            /* Outstanding Receivables Ledger */
            <div className="backdrop-blur-xl bg-white/5 border border-white/10 rounded-2xl p-6 flex flex-col h-[380px]">
                <div className="flex items-center justify-between pb-4 border-b border-white/10 mb-4">
                    <h3 className="text-md font-semibold text-white flex items-center gap-2">
                        <Receipt className="w-4 h-4 text-rose-400" />
                        Outstanding Dues Ledger ({outstandingClients.length})
                    </h3>
                    <span className="text-[10px] px-2 py-0.5 rounded-full bg-rose-500/10 text-rose-400 border border-rose-500/20 font-semibold font-mono">Backlog dues</span>
                </div>
                <div className="flex-1 overflow-y-auto space-y-3 pr-2 scrollbar-thin">
                    {outstandingClients.map((client, i) => (
                        <div key={i} className="flex items-center justify-between p-3 rounded-xl bg-white/5 border border-white/5 hover:border-white/10 transition-all">
                            <div>
                                <div className="font-semibold text-white text-sm">{client.customer_name}</div>
                                <div className="text-[10px] text-[#94A3B8] mt-0.5">Dues for: {client.invoices.join(', ')}</div>
                            </div>
                            <div className="flex items-center gap-3">

```

```

        <span className="font-mono text-sm font-bold text-rose-400">■{client.balance_due.toLocaleString('en-IN')}

```

```

{
  /* --- RECENT INVOICES LEDGER TIMELINE --- */
  <div className="backdrop-blur-xl bg-white/5 border border-white/10 rounded-2xl p-6">
    <div className="flex items-center justify-between pb-4 border-b border-white/10 mb-4">
      <h3 className="text-md font-semibold text-white flex items-center gap-2">
        <Receipt className="w-4 h-4 text-[#0EA5E9]" />
        Recent Billing Transactions
      </h3>
      <button
        onClick={() => setActiveSection('billing')}
        className="text-xs font-semibold text-[#0EA5E9] hover:text-[#0284C7] transition-all flex items-center gap-1 hover:underline"
      >
        Go to Billing panel <ArrowRight className="w-3.5 h-3.5" />
      </button>
    </div>

    <div className="overflow-x-auto">
      <table className="w-full text-left text-sm border-collapse">
        <thead>
          <tr className="border-b border-white/10 text-xs text-[#94A3B8] uppercase">
            <th className="py-3 px-4 font-semibold">Document No</th>
            <th className="py-3 px-4 font-semibold">Customer</th>
            <th className="py-3 px-4 font-semibold">Date</th>
            <th className="py-3 px-4 font-semibold">Doc Type</th>
            <th className="py-3 px-4 font-semibold text-right">Grand Total</th>
            <th className="py-3 px-4 font-semibold text-right">Dues Left</th>
          </tr>
        </thead>
        <tbody className="divide-y divide-white/5">
          {financialInvoices.slice(0, 5).map((inv, idx) => (
            <tr key={idx} className="hover:bg-white/[0.01] transition-colors">
              <td className="py-3 px-4 font-mono font-semibold text-white">{inv.invoice_no}</td>
              <td className="py-3 px-4 text-[#CBD5E1]">{inv.customer_name}</td>
              <td className="py-3 px-4 text-xs text-[#94A3B8]">{inv.date}</td>
              <td className="py-3 px-4">
                {inv.doc_type === 'Quotation' ? (
                  <span className="text-[10px] px-2 py-0.5 rounded-full bg-purple-500/10 text-purple-400 border border-purple-500/20 font-medium">Quote</span>
                ) : (
                  <span className="text-[10px] px-2 py-0.5 rounded-full bg-blue-500/10 text-blue-400 border border-blue-500/20 font-medium">Invoice</span>
                )}
              </td>
              <td className="py-3 px-4 text-right font-mono font-bold text-white">█{(inv.grand_total || 0).toLocaleString('en-IN')}</td>
              <td className="py-3 px-4 text-right">
                {inv.doc_type === 'Quotation' ? (
                  <span className="text-xs text-[#64748B]">~</span>
                ) : (inv.balance_due || 0) <= 0 ? (
                  <span className="text-[10px] px-2 py-0.5 rounded-full bg-emerald-500/10 text-emerald-400 border border-emerald-500/20 font-semibold">Paid</span>
                ) : (
                  <span className="text-[10px] px-2 py-0.5 rounded-full bg-rose-500/10 text-rose-400 border border-rose-500/20 font-semibold font-mono">Due</span>
                )}
              </td>
            </tr>
          ))}
          {financialInvoices.length === 0 && (
            <tr>
              <td colspan={6} className="py-6 text-sm text-[#64748B] italic text-center">No billing transactions recorded yet.</td>
            </tr>
          )}
        </tbody>
      </table>
    </div>
  </div>
);
};

// --- Render Data Table ---
const renderDataTable = () => {
  const config = SECTION_CONFIG[activeSection];
  const columns = SECTION_COLUMNS[activeSection];
  const fields = SECTION_FIELDS[activeSection];
  if (!config || !columns) return null;

  const filteredData = getFilteredData();
  const isReadOnly = activeSection === 'contacts';

  return (
    <div>
      {/* Header */}
      <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4 mb-6">
        <h2 className="text-2xl font-bold text-white">{config.label}</h2>
        <div className="flex flex-wrap items-center gap-3">
          {activeSection === 'tickets' && (
            <a
              href="/service-request"
              target="_blank"
              rel="noreferrer"
              className="inline-flex items-center gap-2 px-4 py-2.5 rounded-lg bg-emerald-500 hover:bg-emerald-600 text-white text-sm font-medium transition"
            >
              <TicketIcon />
              Tickets
            </a>
          )}
        </div>
      </div>
    </div>
  );
};

```

```

    >
    <ExternalLink className="w-4 h-4" />
    Open Customer Form
  </a>
  <button
    onClick={syncFromGoogleSheet}
    disabled={isSyncing}
    className="inline-flex items-center gap-2 px-4 py-2.5 rounded-lg bg-white/5 border border-white/10 text-[#94A3B8] hover:text-white hover:bor
  >
    {isSyncing ? <Loader2 className="w-4 h-4 animate-spin" /> : <RefreshCw className="w-4 h-4" />}
    Sync Old Google Form
  </button>
  <button
    onClick={exportServiceTicketsExcel}
    disabled={isExportingTickets}
    className="inline-flex items-center gap-2 px-4 py-2.5 rounded-lg bg-white/5 border border-white/10 text-[#94A3B8] hover:text-white hover:bor
  >
    {isExportingTickets ? <Loader2 className="w-4 h-4 animate-spin" /> : <FileSpreadsheet className="w-4 h-4" />}
    Export Excel
  </button>
</>
)}
{fields && !isReadOnly && (
  <button
    onClick={openAddForm}
    className="inline-flex items-center gap-2 px-4 py-2.5 rounded-lg bg-[#0EA5E9] hover:bg-[#0284C7] text-white text-sm font-medium transition-all
  >
    <Plus className="w-4 h-4" /> Add New
  </button>
)}
<button
  onClick={() => fetchData(activeSection)}
  className="inline-flex items-center gap-2 px-3 py-2.5 rounded-lg bg-white/5 border border-white/10 text-[#94A3B8] hover:text-white hover:border-
  >
    <RefreshCw className="w-4 h-4" />
  </button>
</div>
</div>

{/* Search */}
<div className="relative mb-4">
  <Search className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-[#64748B]" />
  <input
    type="text"
    placeholder="Search..."
    value={searchQuery}
    onChange={e => setSearchQuery(e.target.value)}
    className="w-full pl-10 pr-4 py-2.5 rounded-lg bg-white/5 border border-white/10 text-white placeholder-[#64748B] text-sm focus:outline-none focus
  />
</div>

{/* Table */}
{dataLoading ? (
  <div className="flex items-center justify-center py-20">
    <Loader2 className="w-6 h-6 text-[#0EA5E9] animate-spin" />
  </div>
) : filteredData.length === 0 ? (
  <div className="text-center py-20 text-[#64748B]">
    <p className="text-lg">No items found</p>
    <p className="text-sm mt-1">Click "Add New" to create your first item</p>
  </div>
) : (
  <div className="overflow-x-auto backdrop-blur-xl bg-white/5 border border-white/10 rounded-xl">
    <table className="w-full">
      <thead>
        <tr className="border-b border-white/10">
          {columns.map(col => (
            <th key={col.key} className="px-4 py-3 text-left text-xs font-semibold text-[#94A3B8] uppercase tracking-wider">
              {col.label}
            </th>
          ))}
        <th className="px-4 py-3 text-right text-xs font-semibold text-[#94A3B8] uppercase tracking-wider">
          Actions
        </th>
      </thead>
      <tbody>
        {filteredData.map((item) => (
          <tr key={String(item.id)} className="border-b border-white/5 hover:bg-white/[0.02] transition-colors">
            {columns.map(col => (
              <td key={col.key} className="px-4 py-3">
                {renderCellValue(item, col.key)}
              </td>
            ))}
            <td className="px-4 py-3">
              <div className="flex items-center justify-end gap-2">
                {config.publishedField && (
                  <button

```



```

onClick={() => togglePublished(item)}
className="p-1.5 rounded-lg hover:bg-white/5 text-[#64748B] hover:text-[#0EA5E9] transition-all"
title={item[config.publishedField] ? 'Unpublish' : 'Publish'}
>
{item[config.publishedField] ? <Eye className="w-4 h-4" /> : <EyeOff className="w-4 h-4" />}
</button>
)}
{activeSection === 'contacts' && (
  <select
    value={String(item.status || 'new')}
    onChange={e => updateContactStatus(String(item.id), e.target.value)}
    className="px-2 py-1 rounded-lg bg-white/5 border border-white/10 text-[#94A3B8] text-xs focus:outline-none focus:border-[#0EA5E9]"
  >
    <option value="new">New</option>
    <option value="read">Read</option>
    <option value="replied">Replied</option>
  </select>
)}
{!isReadOnly && (
  <>
    {activeSection === 'tickets' && (
      <>
        <button
          onClick={() => handleInstantBill(item)}
          className="p-1.5 rounded-lg hover:bg-white/5 text-[#64748B] hover:text-[#0EA5E9] transition-all"
          title="Create Invoice for this Repair"
        >
          <CreditCard className="w-4 h-4" />
        </button>
        <button
          onClick={() => sendWhatsAppAlert(item)}
          className="p-1.5 rounded-lg hover:bg-white/5 text-[#64748B] hover:text-green-400 transition-all"
          title="Send WhatsApp Client Alert"
        >
          <MessageSquare className="w-4 h-4" />
        </button>
        <button
          onClick={() => sendTicketEmail(item)}
          className="p-1.5 rounded-lg hover:bg-white/5 text-[#64748B] hover:text-blue-400 transition-all"
          title="Send Email to Customer"
        >
          <Mail className="w-4 h-4" />
        </button>
        <button
          onClick={() => printJobSheet(item)}
          className="p-1.5 rounded-lg hover:bg-white/5 text-[#64748B] hover:text-emerald-400 transition-all"
          title="Print Job Sheet / Drop-off Receipt"
        >
          <Receipt className="w-4 h-4" />
        </button>
        <button
          onClick={() => printLabel(item)}
          className="p-1.5 rounded-lg hover:bg-white/5 text-[#64748B] hover:text-purple-400 transition-all"
          title="Print Device Sticker Label (QR)"
        >
          <QrCode className="w-4 h-4" />
        </button>
      </>
    )}
  <button
    onClick={() => openEditForm(item)}
    className="p-1.5 rounded-lg hover:bg-white/5 text-[#64748B] hover:text-[#0EA5E9] transition-all"
    title="Edit"
  >
    <Pencil className="w-4 h-4" />
  </button>
  {deleteConfirm === item.id ? (
    <div className="flex items-center gap-1">
      <button
        onClick={() => handleDelete(String(item.id))}
        className="px-2 py-1 rounded-lg bg-red-500/20 text-red-400 text-xs font-medium hover:bg-red-500/30 transition-all"
      >
        Confirm
      </button>
      <button
        onClick={() => setDeleteConfirm(null)}
        className="px-2 py-1 rounded-lg bg-white/5 text-[#94A3B8] text-xs hover:bg-white/10 transition-all"
      >
        Cancel
      </button>
    </div>
  ) : (
    <button
      onClick={() => setDeleteConfirm(String(item.id))}
      className="p-1.5 rounded-lg hover:bg-white/5 text-[#64748B] hover:text-red-400 transition-all"
      title="Delete"
    >
      <Trash2 className="w-4 h-4" />
    </button>
  )}
  </>
)}

```

```

        </button>
      )}
    </>
  )}
</div>
</td>
</tr>
)}}
</tbody>
</table>
</div>
)}
</div>
);
};

// --- Render Settings ---
const renderSettings = () => {
  return (
    <div>
      <div className="flex items-center justify-between mb-6">
        <h2 className="text-2xl font-bold text-white">Site Settings</h2>
        <button
          onClick={saveSettings}
          disabled={!formSaving}
          className="inline-flex items-center gap-2 px-4 py-2.5 rounded-lg bg-[#0EA5E9] hover:bg-[#0284C7] text-white text-sm font-medium transition-all duration-300"
        >
          {formSaving ? <Loader2 className="w-4 h-4 animate-spin" /> : <Save className="w-4 h-4" />}
          Save All
        </button>
      </div>

      {/* Add new setting */}
      <div className="backdrop-blur-xl bg-white/5 border border-white/10 rounded-xl p-4 mb-6">
        <h3 className="text-white text-sm font-semibold mb-3">Add New Setting</h3>
        <div className="flex flex-col sm:flex-row gap-3">
          <input
            type="text"
            placeholder="Key (e.g. company_name)"
            value={newSettingKey}
            onChange={e => setNewSettingKey(e.target.value)}
            className="flex-1 px-3 py-2 rounded-lg bg-white/5 border border-white/10 text-white placeholder-[#64748B] text-sm focus:outline-none focus:border-[#0EA5E9]"
          />
          <input
            type="text"
            placeholder="Value"
            value={newSettingValue}
            onChange={e => setNewSettingValue(e.target.value)}
            className="flex-1 px-3 py-2 rounded-lg bg-white/5 border border-white/10 text-white placeholder-[#64748B] text-sm focus:outline-none focus:border-[#0EA5E9]"
          />
          <button
            onClick={addSetting}
            className="px-4 py-2 rounded-lg bg-[#0EA5E9] hover:bg-[#0284C7] text-white text-sm font-medium transition-all"
          >
            Add
          </button>
        </div>
      </div>

      {settingsLoading ? (
        <div className="flex items-center justify-center py-20">
          <Loader2 className="w-6 h-6 text-[#0EA5E9] animate-spin" />
        </div>
      ) : (
        <div className="space-y-3">
          {settingsRows.map(row => (
            <div key={row.id} className="backdrop-blur-xl bg-white/5 border border-white/10 rounded-xl p-4 flex flex-col sm:flex-row items-start sm:items-center gap-3">
              <div className="w-48 shrink-0">
                <span className="text-[#0EA5E9] text-sm font-mono font-medium">{row.key}</span>
              </div>
              <input
                type="text"
                value={settingsForm[row.key] || ''}
                onChange={e => setSettingsForm(prev => ({ ...prev, [row.key]: e.target.value })}
                className="flex-1 w-full px-3 py-2 rounded-lg bg-white/5 border border-white/10 text-white text-sm focus:outline-none focus:border-[#0EA5E9]"
              />
              <button
                onClick={() => deleteSetting(row.id)}
                className="p-2 rounded-lg hover:bg-white/5 text-[#64748B] hover:text-red-400 transition-all shrink-0"
                title="Delete setting"
              >
                <Trash2 className="w-4 h-4" />
              </button>
            </div>
          ))}
          {settingsRows.length === 0 && (
            <p className="text-center py-12 text-[#64748B]">No settings found. Add your first setting above.</p>
          )}
        </div>
      )}
    </div>
  );
};

```

```

    </div>
  )}
</div>
);
};

// --- Render Form Modal ---
const renderFormModal = () => {
  const fields = SECTION_FIELDS[activeSection];
  if (!fields || !showForm) return null;

  return (
    <div className="fixed inset-0 z-50 flex items-center justify-center p-4">
      <div className="absolute inset-0 bg-black/60 backdrop-blur-sm" onClick={() => setShowForm(false)} />
      <div className="relative w-full max-w-2xl max-h-[90vh] overflow-y-auto backdrop-blur-xl bg-[#0F172A] border border-white/10 rounded-2xl shadow-2xl">
        { /* Header */ }
        <div className="sticky top-0 z-10 bg-[#0F172A] border-b border-white/10 px-6 py-4 flex items-center justify-between">
          <h3 className="text-lg font-bold text-white">
            {editingItem ? 'Edit Item' : 'Add New Item'}
          </h3>
          <button
            onClick={() => setShowForm(false)}
            className="p-1.5 rounded-lg hover:bg-white/5 text-[#64748B] hover:text-white transition-all"
          >
            <X className="w-5 h-5" />
          </button>
        </div>

        { /* Form */ }
        <div className="p-6 space-y-5">
          {formError && (
            <div className="flex items-center gap-2 text-red-400 text-sm bg-red-500/10 border border-red-500/20 rounded-xl p-3">
              <AlertCircle className="w-4 h-4 shrink-0" />
              {formError}
            </div>
          )}

          {activeSection === 'tickets' && (
            <div>
              <label className="block text-white text-sm font-medium mb-2">
                Select Existing Customer
              </label>
              <div className="relative">
                <select
                  value=""
                  onChange={e => {
                    const cust = customersList.find(c => c.id === e.target.value);
                    if (cust) {
                      setFormData(prev => ({
                        ...prev,
                        customer_name: cust.name || '',
                        customer_email: cust.email || '',
                        customer_phone: cust.phone || '',
                        customer_address: cust.address || '',
                      }));
                    }
                  }}
                  className="w-full px-4 py-3 rounded-xl bg-white/5 border border-white/10 text-white text-sm focus:outline-none focus:border-[#0EA5E9]/50 ap
                >
                  <option value="" className="bg-[#0F172A]">Select customer...</option>
                  {customersList.map(c => (
                    <option key={c.id} value={c.id} className="bg-[#0F172A]">
                      {c.name}{c.phone ? ` - ${c.phone}` : ''}
                    </option>
                  ))}
                </select>
                <ChevronDown className="absolute right-3 top-1/2 -translate-y-1/2 w-4 h-4 text-[#64748B] pointer-events-none" />
              </div>
            </div>
          )}

          {fields.map(field => (
            <div key={field.key}>
              <label className="block text-white text-sm font-medium mb-2">
                {field.label}
                {field.required && <span className="text-red-400 ml-1">*</span>}
              </label>

              {field.type === 'text' && (
                <input
                  type="text"
                  value={String(formData[field.key] || '')}
                  onChange={e => setFormData(prev => ({ ...prev, [field.key]: e.target.value })))}
                  placeholder={field.placeholder}
                  className="w-full px-4 py-3 rounded-xl bg-white/5 border border-white/10 text-white placeholder-[#64748B] text-sm focus:outline-none focus
                />
              )}
              {field.type === 'textarea' && (

```

```

<textarea
  value={String(formData[field.key] || '')}
  onChange={e => setFormData(prev => ({ ...prev, [field.key]: e.target.value })))}
  placeholder={field.placeholder}
  rows={field.rows || 4}
  className="w-full px-4 py-3 rounded-xl bg-white/5 border border-white/10 text-white placeholder-[#64748B] text-sm focus:outline-none focus
  />
)}

{field.type === 'number' && (
  <input
    type="number"
    value={String(formData[field.key] || 0)}
    onChange={e => setFormData(prev => ({ ...prev, [field.key]: Number(e.target.value) })))}
    className="w-full px-4 py-3 rounded-xl bg-white/5 border border-white/10 text-white placeholder-[#64748B] text-sm focus:outline-none focus
    />
  )}

{field.type === 'checkbox' && (
  <label className="flex items-center gap-3 cursor-pointer">
    <div
      className={`w-5 h-5 rounded border-2 flex items-center justify-center transition-all ${
        formData[field.key]
          ? 'bg-[#0EA5E9] border-[#0EA5E9]'
          : 'border-white/20 hover:border-[#0EA5E9]/50'
      }`}
      onClick={() => setFormData(prev => ({ ...prev, [field.key]: !prev[field.key] })))}
    >
      {!!formData[field.key] && <CheckCircle className="w-3.5 h-3.5 text-white" />}
    </div>
    <span className="text-[#94A3B8] text-sm">Enable {field.label}</span>
  </label>
)}

{field.type === 'select' && field.options && (
  <div className="relative">
    <select
      value={String(formData[field.key] || '')}
      onChange={e => setFormData(prev => ({ ...prev, [field.key]: e.target.value })))}
      className="w-full px-4 py-3 rounded-xl bg-white/5 border border-white/10 text-white text-sm focus:outline-none focus:border-[#0EA5E9]/50
      >
        <option value="" className="bg-[#0F172A]">Select...</option>
        {field.options.map(opt => (
          <option key={opt.value} value={opt.value} className="bg-[#0F172A]">{opt.label}</option>
        ))}
      </select>
      <ChevronDown className="absolute right-3 top-1/2 -translate-y-1/2 w-4 h-4 text-[#64748B] pointer-events-none" />
    </div>
  )}

{field.type === 'json' && (
  <textarea
    value={String(formData[field.key] || '{}')}
    onChange={e => setFormData(prev => ({ ...prev, [field.key]: e.target.value })))}
    placeholder="{}"
    rows={field.rows || 8}
    className="w-full px-4 py-3 rounded-xl bg-white/5 border border-white/10 text-white placeholder-[#64748B] text-sm focus:outline-none focus
    />
  )}

{field.type === 'array' && (
  <textarea
    value={String(formData[field.key] || '')}
    onChange={e => setFormData(prev => ({ ...prev, [field.key]: e.target.value })))}
    placeholder="One item per line"
    rows={4}
    className="w-full px-4 py-3 rounded-xl bg-white/5 border border-white/10 text-white placeholder-[#64748B] text-sm focus:outline-none focus
    />
  )}
</div>
)))}
</div>

/* Footer */
<div className="sticky bottom-0 bg-[#0F172A] border-t border-white/10 px-6 py-4 flex items-center justify-end gap-3">
  <button
    onClick={() => setShowForm(false)}
    className="px-4 py-2.5 rounded-lg bg-white/5 border border-white/10 text-[#94A3B8] hover:text-white text-sm font-medium transition-all"
  >
    Cancel
  </button>
  <button
    onClick={handleSave}
    disabled={formSaving}
    className="inline-flex items-center gap-2 px-4 py-2.5 rounded-lg bg-[#0EA5E9] hover:bg-[#0284C7] text-white text-sm font-medium transition-all d
  >
    {formSaving ? <Loader2 className="w-4 h-4 animate-spin" /> : <Save className="w-4 h-4" />}
    {formSaving ? 'Saving...' : 'Save'}
  </button>
</div>

```

```

        </button>
      </div>
    </div>
  </div>
);
};

// --- Render content ---
const renderContent = () => {
  if (activeSection === 'dashboard') return renderDashboard();
  if (activeSection === 'settings') return renderSettings();
  if (activeSection === 'billing') return <BillingSoftware initialAutofillTicket={autofillTicket} onClearAutofill={() => setAutofillTicket(null)} />;
  if (activeSection === 'purchase') return <PurchaseSoftware />;
  if (activeSection === 'external-repairs') return <ExternalRepairs />;
  return renderDataTable();
};

// --- Main Layout ---
return (
  <div className="min-h-screen bg-[#0B1120] flex">
    { /* Toast */ }
    <toast && (
      <div className={`fixed top-4 right-4 z-[100] flex items-center gap-2 px-4 py-3 rounded-xl text-sm font-medium shadow-lg transition-all ${
        toast.type === 'success'
          ? 'bg-emerald-500/10 text-emerald-400 border border-emerald-500/20'
          : 'bg-red-500/10 text-red-400 border border-red-500/20'
      }`>
        <{toast.type === 'success' ? <CheckCircle className="w-4 h-4" /> : <AlertCircle className="w-4 h-4" />}
        {toast.message}
      </div>
    )>

    { /* Form Modal */ }
    {renderFormModal()}

    { /* Sidebar */ }
    <aside className={`fixed lg:static inset-y-0 left-0 z-40 w-64 bg-[#0F172A] border-r border-white/10 flex flex-col transition-transform duration-300 ${
      sidebarOpen ? 'translate-x-0' : 'translate-x-[-100%]'
    }`} >
      { /* Logo */ }
      <div className="px-4 py-5 border-b border-white/10 flex items-center gap-3">
        <div className="w-9 h-9 rounded-lg bg-[#0EA5E9]/10 flex items-center justify-center shrink-0">
          <LayoutDashboard className="w-5 h-5 text-[#0EA5E9]" />
        </div>
        {sidebarOpen && (
          <div className="min-w-0">
            <h1 className="text-white font-bold text-sm truncate">Yantrabyte</h1>
            <p className="text-[#64748B] text-xs truncate">Admin Panel</p>
          </div>
        )}
      </div>

      { /* Nav */ }
      <nav className="flex-1 overflow-y-auto py-3 px-2 space-y-0.5">
        {sidebarItems.map(item => {
          const Icon = item.icon;
          const isActive = activeSection === item.section;
          return (
            <button
              key={item.section}
              onClick={() => { setActiveSection(item.section); setSearchQuery(''); setShowForm(false); }}
              className={`w-full flex items-center gap-3 px-3 py-2.5 rounded-lg text-sm font-medium transition-all ${
                isActive
                  ? 'bg-[#0EA5E9]/10 text-[#0EA5E9] border border-[#0EA5E9]/20'
                  : 'text-[#94A3B8] hover:bg-white/5 hover:text-white border border-transparent'
              }`}
            >
              <Icon className="w-4.5 h-4.5 shrink-0" />
              {sidebarOpen && <span className="truncate">{item.label}</span>}
            </button>
          );
        })}
      </nav>

      { /* Logout */ }
      <div className="px-2 py-3 border-t border-white/10">
        <button
          onClick={handleLogout}
          className="w-full flex items-center gap-3 px-3 py-2.5 rounded-lg text-sm font-medium text-[#94A3B8] hover:bg-red-500/10 hover:text-red-400 transition-all"
        >
          <Logout className="w-4.5 h-4.5 shrink-0" />
          {sidebarOpen && <span>Sign Out</span>}
        </button>
      </div>
    </aside>

    { /* Mobile overlay */ }
    {sidebarOpen && (
      <div
        className="fixed inset-0 z-30 bg-black/40 lg:hidden"

```

```

        onClick={() => setSidebarOpen(false)}
      />
    )}

    { /* Main Content */ }
    <main className="flex-1 min-w-0">
      { /* Top Bar */ }
      <header className="sticky top-0 z-20 bg-[#0B1120]/80 backdrop-blur-xl border-b border-white/10 px-4 lg:px-6 py-3 flex items-center justify-between">
        <div className="flex items-center gap-3">
          <button
            onClick={() => setSidebarOpen(!sidebarOpen)}
            className="p-2 rounded-lg hover:bg-white/5 text-[#94A3B8] hover:text-white transition-all lg:hidden"
          >
            <Menu className="w-5 h-5" />
          </button>
          <div>
            <h2 className="text-white font-semibold text-lg">
              {SECTION_CONFIG[activeSection]?.label || 'Dashboard'}
            </h2>
            <p className="text-[#64748B] text-xs">Manage your website content</p>
          </div>
        </div>
        <div className="flex items-center gap-3">
          <button
            onClick={handleLogout}
            className="inline-flex items-center gap-2 px-3 py-2 rounded-lg text-sm text-[#94A3B8] hover:text-red-400 hover:bg-red-500/10 transition-all"
          >
            <LogOut className="w-4 h-4" />
            <span className="hidden sm:inline">Sign Out</span>
          </button>
        </div>
      </header>

      { /* Content Area */ }
      <div className="p-4 lg:p-6">
        {renderContent()}
      </div>
    </main>
  </div>
);
}

```

PART 3: SUPABASE DATABASE MIGRATIONS (SQL)

SQL Migration Scripts to set up schemas, triggers, RLS, and functions in PostgreSQL.

Migration file: 20260508113256_001_initial_schema.sql

```
/*
# Yantrabyte Solutions - Initial Database Schema

1. New Tables
- `pages` - Dynamic page management (title, slug, content, is_published, page_order)
- `services` - Service listings (title, slug, description, icon, features, is_published, sort_order)
- `products` - Product listings (name, slug, description, category, price, image_url, is_featured, is_published, sort_order)
- `testimonials` - Customer testimonials (name, company, rating, content, avatar_url, is_published, sort_order)
- `blog_posts` - Blog articles (title, slug, excerpt, content, category, featured_image, is_published, published_at, sort_order)
- `team_members` - Team members (name, role, bio, image_url, is_published, sort_order)
- `careers` - Job listings (title, slug, description, requirements, location, type, is_active, sort_order)
- `industries` - Industries served (name, slug, description, icon, is_published, sort_order)
- `faqs` - FAQ entries (question, answer, category, is_published, sort_order)
- `site_settings` - Global site settings (key, value pairs)
- `contact_submissions` - Contact form submissions (name, email, phone, service, message, status)
- `gallery_images` - Before/after gallery (title, before_url, after_url, category, is_published, sort_order)
- `client_logos` - Client logo carousel (name, logo_url, is_published, sort_order)

2. Security
- Enable RLS on all tables
- Public read access for published content
- Authenticated admin access for all operations
*/

-- Pages table
CREATE TABLE IF NOT EXISTS pages (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  title text NOT NULL,
  slug text UNIQUE NOT NULL,
  content jsonb DEFAULT '{}',
  meta_title text DEFAULT '',
  meta_description text DEFAULT '',
  is_published boolean DEFAULT false,
  page_order integer DEFAULT 0,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- Services table
CREATE TABLE IF NOT EXISTS services (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  title text NOT NULL,
  slug text UNIQUE NOT NULL,
  short_description text DEFAULT '',
  full_description text DEFAULT '',
  icon text DEFAULT '',
  features jsonb DEFAULT '[]',
  benefits jsonb DEFAULT '[]',
  meta_title text DEFAULT '',
  meta_description text DEFAULT '',
  is_featured boolean DEFAULT false,
  is_published boolean DEFAULT true,
  sort_order integer DEFAULT 0,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- Products table
CREATE TABLE IF NOT EXISTS products (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  slug text UNIQUE NOT NULL,
  description text DEFAULT '',
  category text DEFAULT '',
  price text DEFAULT '',
  image_url text DEFAULT '',
  features jsonb DEFAULT '[]',
  is_featured boolean DEFAULT false,
  is_published boolean DEFAULT true,
  sort_order integer DEFAULT 0,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- Testimonials table
CREATE TABLE IF NOT EXISTS testimonials (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

name text NOT NULL,
company text DEFAULT '',
role text DEFAULT '',
rating integer DEFAULT 5,
content text NOT NULL,
avatar_url text DEFAULT '',
is_published boolean DEFAULT true,
sort_order integer DEFAULT 0,
created_at timestamptz DEFAULT now()
);

-- Blog posts table
CREATE TABLE IF NOT EXISTS blog_posts (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  title text NOT NULL,
  slug text UNIQUE NOT NULL,
  excerpt text DEFAULT '',
  content text DEFAULT '',
  category text DEFAULT '',
  featured_image text DEFAULT '',
  meta_title text DEFAULT '',
  meta_description text DEFAULT '',
  is_published boolean DEFAULT false,
  published_at timestamptz DEFAULT now(),
  sort_order integer DEFAULT 0,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- Team members table
CREATE TABLE IF NOT EXISTS team_members (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  role text DEFAULT '',
  bio text DEFAULT '',
  image_url text DEFAULT '',
  is_published boolean DEFAULT true,
  sort_order integer DEFAULT 0,
  created_at timestamptz DEFAULT now()
);

-- Careers table
CREATE TABLE IF NOT EXISTS careers (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  title text NOT NULL,
  slug text UNIQUE NOT NULL,
  description text DEFAULT '',
  requirements jsonb DEFAULT '[]',
  location text DEFAULT 'Bangalore',
  type text DEFAULT 'Full-time',
  is_active boolean DEFAULT true,
  sort_order integer DEFAULT 0,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- Industries table
CREATE TABLE IF NOT EXISTS industries (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  slug text UNIQUE NOT NULL,
  description text DEFAULT '',
  icon text DEFAULT '',
  is_published boolean DEFAULT true,
  sort_order integer DEFAULT 0,
  created_at timestamptz DEFAULT now()
);

-- FAQs table
CREATE TABLE IF NOT EXISTS faqs (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  question text NOT NULL,
  answer text NOT NULL,
  category text DEFAULT 'general',
  is_published boolean DEFAULT true,
  sort_order integer DEFAULT 0,
  created_at timestamptz DEFAULT now()
);

-- Site settings table
CREATE TABLE IF NOT EXISTS site_settings (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  key text UNIQUE NOT NULL,
  value text DEFAULT '',
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- Contact submissions table

```



```

CREATE TABLE IF NOT EXISTS contact_submissions (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  email text NOT NULL,
  phone text DEFAULT '',
  service text DEFAULT '',
  message text DEFAULT '',
  status text DEFAULT 'new',
  created_at timestamptz DEFAULT now()
);

-- Gallery images table
CREATE TABLE IF NOT EXISTS gallery_images (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  title text DEFAULT '',
  before_url text DEFAULT '',
  after_url text DEFAULT '',
  category text DEFAULT '',
  is_published boolean DEFAULT true,
  sort_order integer DEFAULT 0,
  created_at timestamptz DEFAULT now()
);

-- Client logos table
CREATE TABLE IF NOT EXISTS client_logos (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  logo_url text DEFAULT '',
  is_published boolean DEFAULT true,
  sort_order integer DEFAULT 0,
  created_at timestamptz DEFAULT now()
);

-- Enable RLS on all tables
ALTER TABLE pages ENABLE ROW LEVEL SECURITY;
ALTER TABLE services ENABLE ROW LEVEL SECURITY;
ALTER TABLE products ENABLE ROW LEVEL SECURITY;
ALTER TABLE testimonials ENABLE ROW LEVEL SECURITY;
ALTER TABLE blog_posts ENABLE ROW LEVEL SECURITY;
ALTER TABLE team_members ENABLE ROW LEVEL SECURITY;
ALTER TABLE careers ENABLE ROW LEVEL SECURITY;
ALTER TABLE industries ENABLE ROW LEVEL SECURITY;
ALTER TABLE faqs ENABLE ROW LEVEL SECURITY;
ALTER TABLE site_settings ENABLE ROW LEVEL SECURITY;
ALTER TABLE contact_submissions ENABLE ROW LEVEL SECURITY;
ALTER TABLE gallery_images ENABLE ROW LEVEL SECURITY;
ALTER TABLE client_logos ENABLE ROW LEVEL SECURITY;

-- Public read policies for published content
CREATE POLICY "Public can view published pages" ON pages FOR SELECT TO anon, authenticated USING (is_published = true);
CREATE POLICY "Public can view published services" ON services FOR SELECT TO anon, authenticated USING (is_published = true);
CREATE POLICY "Public can view published products" ON products FOR SELECT TO anon, authenticated USING (is_published = true);
CREATE POLICY "Public can view published testimonials" ON testimonials FOR SELECT TO anon, authenticated USING (is_published = true);
CREATE POLICY "Public can view published blog posts" ON blog_posts FOR SELECT TO anon, authenticated USING (is_published = true);
CREATE POLICY "Public can view published team members" ON team_members FOR SELECT TO anon, authenticated USING (is_published = true);
CREATE POLICY "Public can view active careers" ON careers FOR SELECT TO anon, authenticated USING (is_active = true);
CREATE POLICY "Public can view published industries" ON industries FOR SELECT TO anon, authenticated USING (is_published = true);
CREATE POLICY "Public can view published FAQs" ON faqs FOR SELECT TO anon, authenticated USING (is_published = true);
CREATE POLICY "Public can view site settings" ON site_settings FOR SELECT TO anon, authenticated USING (true);
CREATE POLICY "Public can view published gallery" ON gallery_images FOR SELECT TO anon, authenticated USING (is_published = true);
CREATE POLICY "Public can view published client logos" ON client_logos FOR SELECT TO anon, authenticated USING (is_published = true);
CREATE POLICY "Public can submit contact forms" ON contact_submissions FOR INSERT TO anon, authenticated WITH CHECK (true);

-- Admin full access policies (authenticated users)
CREATE POLICY "Admin full access pages" ON pages FOR ALL TO authenticated USING (true) WITH CHECK (true);
CREATE POLICY "Admin full access services" ON services FOR ALL TO authenticated USING (true) WITH CHECK (true);
CREATE POLICY "Admin full access products" ON products FOR ALL TO authenticated USING (true) WITH CHECK (true);
CREATE POLICY "Admin full access testimonials" ON testimonials FOR ALL TO authenticated USING (true) WITH CHECK (true);
CREATE POLICY "Admin full access blog_posts" ON blog_posts FOR ALL TO authenticated USING (true) WITH CHECK (true);
CREATE POLICY "Admin full access team_members" ON team_members FOR ALL TO authenticated USING (true) WITH CHECK (true);
CREATE POLICY "Admin full access careers" ON careers FOR ALL TO authenticated USING (true) WITH CHECK (true);
CREATE POLICY "Admin full access industries" ON industries FOR ALL TO authenticated USING (true) WITH CHECK (true);
CREATE POLICY "Admin full access faqs" ON faqs FOR ALL TO authenticated USING (true) WITH CHECK (true);
CREATE POLICY "Admin full access site_settings" ON site_settings FOR ALL TO authenticated USING (true) WITH CHECK (true);
CREATE POLICY "Admin full access contact_submissions" ON contact_submissions FOR ALL TO authenticated USING (true) WITH CHECK (true);
CREATE POLICY "Admin full access gallery_images" ON gallery_images FOR ALL TO authenticated USING (true) WITH CHECK (true);
CREATE POLICY "Admin full access client_logos" ON client_logos FOR ALL TO authenticated USING (true) WITH CHECK (true);

-- Insert default site settings
INSERT INTO site_settings (key, value) VALUES
  ('company_name', 'Yantrabyte Solutions'),
  ('tagline', 'Smart Technology. Secure Future.'),
  ('phone', '+91-9876543210'),
  ('email', 'info@yantrabyte.com'),
  ('whatsapp', '919876543210'),
  ('address', 'Bangalore, Karnataka, India'),
  ('google_maps_embed', ''),
  ('facebook', '');

```

```

('instagram', ''),
('linkedin', ''),
('twitter', ''),
('youtube', '');

-- Insert default services
INSERT INTO services (title, slug, short_description, icon, features, is_featured, is_published, sort_order) VALUES
('CCTV Installation Services', 'cctv-installation-bangalore', 'Professional CCTV camera installation for homes and businesses across Bangalore with HD and 4K resolution.', 'CCTVIcon', true, 1),
('IP Camera Installation', 'ip-camera-installation-bangalore', 'Advanced IP camera solutions with remote monitoring, cloud storage, and AI-powered analytics.', 'IPCameraIcon', true, 2),
('Laptop Repair Services', 'laptop-repair-bangalore', 'Expert laptop repair for all brands – screen replacement, motherboard repair, data recovery, and more.', 'LaptopIcon', true, 3),
('Desktop Repair Services', 'desktop-repair-bangalore', 'Complete desktop computer repair including hardware upgrades, virus removal, and performance optimization.', 'DesktopIcon', true, 4),
('Printer Repair & Maintenance', 'printer-repair-bangalore', 'Printer repair and maintenance for HP, Canon, Epson, Brother, and all major brands.', 'PrinterIcon', true, 5),
('WiFi & Networking Solutions', 'networking-solutions-bangalore', 'Complete WiFi and networking solutions for offices, homes, and commercial spaces in Bangalore.', 'WiFiIcon', true, 6),
('Biometric Attendance Systems', 'biometric-installation-bangalore', 'Fingerprint, face recognition, and RFID-based attendance and access control systems.', 'BiometricIcon', true, 7),
('Firewall & Network Security', 'firewall-network-security-bangalore', 'Enterprise-grade firewall installation, network security audits, and cyber threat protection.', 'SecurityIcon', true, 8),
('Office IT Setup', 'office-it-setup-bangalore', 'Complete office IT infrastructure setup including computers, networking, printers, and software.', 'OfficeIcon', true, 9),
('AMC Support Services', 'amc-services-bangalore', 'Annual Maintenance Contracts for IT equipment, CCTV systems, and office technology.', 'FileCheck', ["Regular maintenance visits", "Priority support", "Spare parts (select plans)", "Performance reports", "Emergency response"], true, 10),
('Server Installation', 'server-installation-bangalore', 'Server setup, configuration, and maintenance for businesses of all sizes.', 'Server', ["Server hardware", "Network configuration", "Data backup solutions", "24/7 monitoring"], true, 11),
('Remote IT Support', 'remote-it-support-bangalore', 'Instant remote IT support for troubleshooting, software issues, and system maintenance.', 'Headphones', true, 12),
('Home Security Solutions', 'home-security-bangalore', 'Complete home security packages with CCTV, video door phones, and smart locks.', 'Home', ["CCTV cameras", "Video door phones", "Smart locks", "Alarm systems"], true, 13),
('Smart Office Automation', 'smart-office-automation-bangalore', 'Automate your office with smart lighting, climate control, and IoT solutions.', 'Zap', ["Smart lighting", "Climate control", "IoT integration"], true, 14),
('Access Control Systems', 'access-control-bangalore', 'Card-based, biometric, and mobile access control systems for offices and buildings.', 'KeyRound', ["Access cards", "Biometric readers", "Mobile app access"], true, 15);

-- Insert default industries
INSERT INTO industries (name, slug, description, icon, is_published, sort_order) VALUES
('Corporate Offices', 'corporate-offices', 'Complete IT infrastructure and security solutions for modern corporate offices.', 'Building', true, 1),
('Retail Stores', 'retail-stores', 'CCTV surveillance, POS systems, and networking for retail businesses.', 'ShoppingCart', true, 2),
('Warehouses', 'warehouses', 'Security cameras, access control, and network infrastructure for warehouses.', 'Warehouse', true, 3),
('Apartments & Residences', 'apartments-residences', 'Home security, WiFi, and IT support for residential complexes.', 'Home', true, 4),
('Schools & Colleges', 'schools-colleges', 'Campus security, IT labs, and networking for educational institutions.', 'GraduationCap', true, 5),
('Hospitals & Clinics', 'hospitals-clinics', 'Healthcare IT, security systems, and biometric attendance for medical facilities.', 'HeartPulse', true, 6),
('Factories & Manufacturing', 'factories-manufacturing', 'Industrial security, networking, and IT support for manufacturing units.', 'Factory', true, 7),
('Homes', 'homes', 'Smart home security, WiFi setup, and computer repair for residences.', 'House', true, 8);

-- Insert default testimonials
INSERT INTO testimonials (name, company, role, rating, content, is_published, sort_order) VALUES
('Rajesh Kumar', 'TechCorp Solutions', 'Operations Manager', 5, 'Yantrabyte installed CCTV across our 3 office floors. The quality is outstanding and their support is excellent.', true, 1),
('Priya Sharma', 'RetailMax India', 'Owner', 5, 'Best laptop repair service in Bangalore! They fixed my MacBook the same day at a very reasonable price. Highly recommended!', true, 2),
('Mohammed Irfan', 'Green Valley Apartments', 'Secretary', 5, 'Complete WiFi and CCTV setup for our 200-apartment complex. Excellent work and great after-sales service.', true, 3),
('Dr. Anitha Rao', 'MediCare Clinic', 'Director', 5, 'Their biometric attendance system integrated perfectly with our payroll. The team understood our requirements and delivered on time.', true, 4),
('Vikram Patel', 'Patel Industries', 'MD', 5, 'We have been using their AMC service for 2 years now. Response time is excellent and they keep all our IT systems secure.', true, 5);

-- Insert default FAQs
INSERT INTO faqs (question, answer, category, is_published, sort_order) VALUES
('What areas in Bangalore do you serve?', 'We serve all areas in Bangalore including Whitefield, Electronic City, Koramangala, Indiranagar, HSR Layout, Marathahalli, and more.', 'Coverage', true, 1),
('Do you provide same-day service?', 'Yes! We offer same-day service for most repair and installation requests. Emergency support is available 24/7.', 'Service', true, 2),
('What brands of CCTV cameras do you install?', 'We install all major brands including Hikvision, Dahua, CP Plus, Godrej, and Samsung. We recommend the best quality for your budget.', 'Products', true, 3),
('How much does CCTV installation cost?', 'CCTV installation costs vary based on the number of cameras, type (analog/IP), and features required. Contact us for a free quote.', 'Pricing', true, 4),
('Do you provide warranty on repairs?', 'Yes, we provide a 90-day warranty on all repair services and 1-year warranty on new installations.', 'Warranty', true, 5),
('What is included in AMC support?', 'Our AMC includes regular maintenance visits, priority support, spare parts (select plans), performance reports, and emergency response.', 'AMC', true, 6),
('Can I monitor CCTV from my phone?', 'Absolutely! All our CCTV installations come with mobile app setup so you can monitor your property from anywhere in the world.', 'Monitoring', true, 7),
('Do you offer free consultations?', 'Yes, we offer free on-site consultations for CCTV, networking, and office IT setup projects. Call us to schedule yours.', 'Consultation', true, 8);

-- Insert default blog posts
INSERT INTO blog_posts (title, slug, excerpt, content, category, is_published, published_at, sort_order) VALUES
('Best CCTV Camera for Office in Bangalore 2025', 'best-cctv-camera-office-bangalore', 'A comprehensive guide to choosing the right CCTV cameras for your Bangalore office.', 'CCTV', true, '2025-01-15', 1),
('Laptop Running Slow? Here's the Fix', 'laptop-running-slow-fix', 'Is your laptop crawling? Learn the most common causes and proven fixes to speed up your laptop performance.', 'IT Support', true, '2025-01-10', 2),
('Top Office Networking Solutions in Bangalore', 'top-office-networking-bangalore', 'Discover the best networking solutions for Bangalore offices, from WiFi upgrades to cloud-based networks.', 'Networking', true, '2025-01-05', 3),
('DVR vs NVR Explained: Which is Right for You?', 'dvr-vs-nvr-explained', 'Understanding the difference between DVR and NVR systems is crucial for choosing the right security solution.', 'CCTV', true, '2025-01-01', 4),
('How to Improve Business Security in Bangalore', 'improve-business-security-bangalore', 'Essential security measures every Bangalore business should implement to protect their data and assets.', 'Security', true, '2025-01-01', 5);

-- Insert default careers
INSERT INTO careers (title, slug, description, requirements, location, type, is_active, sort_order) VALUES
('CCTV Installation Technician', 'cctv-installation-technician', 'We are looking for experienced CCTV installation technicians to join our growing team in Bangalore.', 'Bangalore', 'Full-time', true, 1),
('IT Support Engineer', 'it-support-engineer', 'Join our IT support team to help businesses across Bangalore with their technology needs.', 'Bangalore', 'Full-time', true, 2),
('Network Engineer', 'network-engineer', 'We need a skilled network engineer to design and implement networking solutions for our clients.', 'Bangalore', 'Full-time', true, 3);

```

[illegible]


```

ON public.contact_submissions FOR UPDATE
TO authenticated
USING (public.is_admin())
WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete contact_submissions"
ON public.contact_submissions FOR DELETE
TO authenticated
USING (public.is_admin());

-- faqs
CREATE POLICY "Admin can view faqs"
ON public.faqs FOR SELECT
TO authenticated
USING (public.is_admin());

CREATE POLICY "Admin can insert faqs"
ON public.faqs FOR INSERT
TO authenticated
WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update faqs"
ON public.faqs FOR UPDATE
TO authenticated
USING (public.is_admin())
WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete faqs"
ON public.faqs FOR DELETE
TO authenticated
USING (public.is_admin());

-- gallery_images
CREATE POLICY "Admin can view gallery_images"
ON public.gallery_images FOR SELECT
TO authenticated
USING (public.is_admin());

CREATE POLICY "Admin can insert gallery_images"
ON public.gallery_images FOR INSERT
TO authenticated
WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update gallery_images"
ON public.gallery_images FOR UPDATE
TO authenticated
USING (public.is_admin())
WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete gallery_images"
ON public.gallery_images FOR DELETE
TO authenticated
USING (public.is_admin());

-- industries
CREATE POLICY "Admin can view industries"
ON public.industries FOR SELECT
TO authenticated
USING (public.is_admin());

CREATE POLICY "Admin can insert industries"
ON public.industries FOR INSERT
TO authenticated
WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update industries"
ON public.industries FOR UPDATE
TO authenticated
USING (public.is_admin())
WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete industries"
ON public.industries FOR DELETE
TO authenticated
USING (public.is_admin());

-- pages
CREATE POLICY "Admin can view pages"
ON public.pages FOR SELECT
TO authenticated
USING (public.is_admin());

CREATE POLICY "Admin can insert pages"
ON public.pages FOR INSERT
TO authenticated
WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update pages"

```

```

ON public.pages FOR UPDATE
TO authenticated
USING (public.is_admin())
WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete pages"
ON public.pages FOR DELETE
TO authenticated
USING (public.is_admin());

-- products
CREATE POLICY "Admin can view products"
ON public.products FOR SELECT
TO authenticated
USING (public.is_admin());

CREATE POLICY "Admin can insert products"
ON public.products FOR INSERT
TO authenticated
WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update products"
ON public.products FOR UPDATE
TO authenticated
USING (public.is_admin())
WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete products"
ON public.products FOR DELETE
TO authenticated
USING (public.is_admin());

-- services
CREATE POLICY "Admin can view services"
ON public.services FOR SELECT
TO authenticated
USING (public.is_admin());

CREATE POLICY "Admin can insert services"
ON public.services FOR INSERT
TO authenticated
WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update services"
ON public.services FOR UPDATE
TO authenticated
USING (public.is_admin())
WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete services"
ON public.services FOR DELETE
TO authenticated
USING (public.is_admin());

-- site_settings
CREATE POLICY "Admin can view site_settings"
ON public.site_settings FOR SELECT
TO authenticated
USING (public.is_admin());

CREATE POLICY "Admin can insert site_settings"
ON public.site_settings FOR INSERT
TO authenticated
WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update site_settings"
ON public.site_settings FOR UPDATE
TO authenticated
USING (public.is_admin())
WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete site_settings"
ON public.site_settings FOR DELETE
TO authenticated
USING (public.is_admin());

-- team_members
CREATE POLICY "Admin can view team_members"
ON public.team_members FOR SELECT
TO authenticated
USING (public.is_admin());

CREATE POLICY "Admin can insert team_members"
ON public.team_members FOR INSERT
TO authenticated
WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update team_members"

```

[illegible]

[illegible]


```

    USING (public.is_admin());

CREATE POLICY "Admin can insert client_logos"
    ON public.client_logos FOR INSERT
    TO authenticated
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update client_logos"
    ON public.client_logos FOR UPDATE
    TO authenticated
    USING (public.is_admin())
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete client_logos"
    ON public.client_logos FOR DELETE
    TO authenticated
    USING (public.is_admin());

-- contact_submissions
CREATE POLICY "Admin can view contact_submissions"
    ON public.contact_submissions FOR SELECT
    TO authenticated
    USING (public.is_admin());

CREATE POLICY "Admin can insert contact_submissions"
    ON public.contact_submissions FOR INSERT
    TO authenticated
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update contact_submissions"
    ON public.contact_submissions FOR UPDATE
    TO authenticated
    USING (public.is_admin())
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete contact_submissions"
    ON public.contact_submissions FOR DELETE
    TO authenticated
    USING (public.is_admin());

-- faqs
CREATE POLICY "Admin can view faqs"
    ON public.faqs FOR SELECT
    TO authenticated
    USING (public.is_admin());

CREATE POLICY "Admin can insert faqs"
    ON public.faqs FOR INSERT
    TO authenticated
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update faqs"
    ON public.faqs FOR UPDATE
    TO authenticated
    USING (public.is_admin())
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete faqs"
    ON public.faqs FOR DELETE
    TO authenticated
    USING (public.is_admin());

-- gallery_images
CREATE POLICY "Admin can view gallery_images"
    ON public.gallery_images FOR SELECT
    TO authenticated
    USING (public.is_admin());

CREATE POLICY "Admin can insert gallery_images"
    ON public.gallery_images FOR INSERT
    TO authenticated
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update gallery_images"
    ON public.gallery_images FOR UPDATE
    TO authenticated
    USING (public.is_admin())
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete gallery_images"
    ON public.gallery_images FOR DELETE
    TO authenticated
    USING (public.is_admin());

-- industries
CREATE POLICY "Admin can view industries"
    ON public.industries FOR SELECT
    TO authenticated

```

```

    USING (public.is_admin());

CREATE POLICY "Admin can insert industries"
    ON public.industries FOR INSERT
    TO authenticated
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update industries"
    ON public.industries FOR UPDATE
    TO authenticated
    USING (public.is_admin())
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete industries"
    ON public.industries FOR DELETE
    TO authenticated
    USING (public.is_admin());

-- pages
CREATE POLICY "Admin can view pages"
    ON public.pages FOR SELECT
    TO authenticated
    USING (public.is_admin());

CREATE POLICY "Admin can insert pages"
    ON public.pages FOR INSERT
    TO authenticated
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update pages"
    ON public.pages FOR UPDATE
    TO authenticated
    USING (public.is_admin())
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete pages"
    ON public.pages FOR DELETE
    TO authenticated
    USING (public.is_admin());

-- products
CREATE POLICY "Admin can view products"
    ON public.products FOR SELECT
    TO authenticated
    USING (public.is_admin());

CREATE POLICY "Admin can insert products"
    ON public.products FOR INSERT
    TO authenticated
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update products"
    ON public.products FOR UPDATE
    TO authenticated
    USING (public.is_admin())
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete products"
    ON public.products FOR DELETE
    TO authenticated
    USING (public.is_admin());

-- services
CREATE POLICY "Admin can view services"
    ON public.services FOR SELECT
    TO authenticated
    USING (public.is_admin());

CREATE POLICY "Admin can insert services"
    ON public.services FOR INSERT
    TO authenticated
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update services"
    ON public.services FOR UPDATE
    TO authenticated
    USING (public.is_admin())
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete services"
    ON public.services FOR DELETE
    TO authenticated
    USING (public.is_admin());

-- site_settings
CREATE POLICY "Admin can view site_settings"
    ON public.site_settings FOR SELECT
    TO authenticated

```

```

    USING (public.is_admin());

CREATE POLICY "Admin can insert site_settings"
    ON public.site_settings FOR INSERT
    TO authenticated
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update site_settings"
    ON public.site_settings FOR UPDATE
    TO authenticated
    USING (public.is_admin())
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete site_settings"
    ON public.site_settings FOR DELETE
    TO authenticated
    USING (public.is_admin());

-- team_members
CREATE POLICY "Admin can view team_members"
    ON public.team_members FOR SELECT
    TO authenticated
    USING (public.is_admin());

CREATE POLICY "Admin can insert team_members"
    ON public.team_members FOR INSERT
    TO authenticated
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update team_members"
    ON public.team_members FOR UPDATE
    TO authenticated
    USING (public.is_admin())
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete team_members"
    ON public.team_members FOR DELETE
    TO authenticated
    USING (public.is_admin());

-- testimonials
CREATE POLICY "Admin can view testimonials"
    ON public.testimonials FOR SELECT
    TO authenticated
    USING (public.is_admin());

CREATE POLICY "Admin can insert testimonials"
    ON public.testimonials FOR INSERT
    TO authenticated
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can update testimonials"
    ON public.testimonials FOR UPDATE
    TO authenticated
    USING (public.is_admin())
    WITH CHECK (public.is_admin());

CREATE POLICY "Admin can delete testimonials"
    ON public.testimonials FOR DELETE
    TO authenticated
    USING (public.is_admin());

```

Migration file: 20260515120000_add_service_tickets.sql

```
-- Create service_tickets table
CREATE TABLE IF NOT EXISTS public.service_tickets (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  ticket_number text UNIQUE NOT NULL,
  customer_name text NOT NULL,
  customer_email text,
  customer_phone text NOT NULL,
  device_type text,
  issue_description text NOT NULL,
  status text NOT NULL DEFAULT 'open',
  priority text NOT NULL DEFAULT 'medium',
  assigned_to uuid REFERENCES auth.users(id),
  notes text,
  created_at timestampz DEFAULT now(),
  updated_at timestampz DEFAULT now()
);

-- Enable RLS
ALTER TABLE public.service_tickets ENABLE ROW LEVEL SECURITY;

-- Create policy for authenticated users (admins) to have full access
CREATE POLICY "Admins have full access to service_tickets"
  ON public.service_tickets
  FOR ALL
  TO authenticated
  USING (true)
  WITH CHECK (true);
```

Migration file: 20260516120000_add_invoices_table.sql

```
-- Create invoices table for the built-in Billing Software
CREATE TABLE IF NOT EXISTS public.invoices (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    invoice_no TEXT UNIQUE NOT NULL,
    doc_type TEXT NOT NULL DEFAULT 'Invoice',
    date TEXT NOT NULL,
    customer_name TEXT NOT NULL,
    phone TEXT,
    email TEXT,
    address TEXT,
    items JSONB NOT NULL DEFAULT '[]'::jsonb,
    subtotal NUMERIC NOT NULL DEFAULT 0,
    discount NUMERIC NOT NULL DEFAULT 0,
    tax NUMERIC NOT NULL DEFAULT 0,
    round_off NUMERIC NOT NULL DEFAULT 0,
    grand_total NUMERIC NOT NULL DEFAULT 0,
    advance_paid NUMERIC NOT NULL DEFAULT 0,
    balance_due NUMERIC NOT NULL DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- RLS Policies
ALTER TABLE public.invoices ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Allow read access for admins"
    ON public.invoices FOR SELECT
    USING (auth.role() = 'authenticated');

CREATE POLICY "Allow insert access for admins"
    ON public.invoices FOR INSERT
    WITH CHECK (auth.role() = 'authenticated');

CREATE POLICY "Allow update access for admins"
    ON public.invoices FOR UPDATE
    USING (auth.role() = 'authenticated');

CREATE POLICY "Allow delete access for admins"
    ON public.invoices FOR DELETE
    USING (auth.role() = 'authenticated');
```

Migration file: 20260516130000_form_responses_1.sql

```
CREATE TABLE IF NOT EXISTS "Form Responses 1" (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  full_name text,  
  email text,  
  phone text,  
  address text,  
  created_at timestamptz DEFAULT now()  
);  
  
-- Enable RLS  
ALTER TABLE "Form Responses 1" ENABLE ROW LEVEL SECURITY;  
  
-- Drop policy if exists to avoid errors on reruns  
DROP POLICY IF EXISTS "Enable read access for authenticated users" ON "Form Responses 1";  
  
-- Create policy for select  
CREATE POLICY "Enable read access for authenticated users" ON "Form Responses 1" FOR SELECT TO authenticated USING (true);
```

Migration file: 20260516140000_insert_form_responses.sql

```
INSERT INTO "Form Responses 1" (full_name, phone, email, address) VALUES
('Manjesh', '8660701899', 'tmanjesh.7712@gmail.com', 'Chamundeswari layout'),
('Dr Sreelekshmi ', '9207199609', 'vidyaranypuraclinicreception@gmail.com', 'Dr Sreelekshmi Kerala Ayurveda Centre Vidyaranyaapura '),
('Girish Malige', '9886253098', 'girimalice@yahoo.co.in', '14, SaNa, Sri Saraswathi Nivasa, 2nd Main, Sri Dhanalakshmi Layout, Virupakshapura '),
('Mallarswami nonvinkere', '9731846846', 'mailar@gmail.com', 'Defence layout vidyaranyaapura '),
('Sagar', '9591451419', 'sagar.venkatesh1989@gmail.com', 'Vidyaranyaapura, bangalore '),
('Girish', '9945408547', 'giri1119@gmail.com', 'Babu house road, 5th cross,Raghavendra colony, Vidyaranyaapura'),
('Pawan', '9901733369', 'YantraByte.solutions@gmail.com', 'Vidyaranyaapura'),
('Iibs', '9901733369', 'rameshas2105@gmail.com', 'Vidyaranyaapura '),
('Akhil Rajput ', '8548035733', 'Andajay200@gmail.com', 'Flat no 201, Durga Nivas,kanshiram nagar 560097, '),
('yashwant', '9901733369', 'rameshas2105@gmail.com', 'ms playa'),
('test', '99999999999', 'rameshas2105@gmail.com', 'ms playa'),
('test ', '9999999999', 'rameshas2105@gmail.com', 'vr pura'),
('test', '9999999999', 'sysadmin@iibsonline.com', 'yelekhanka'),
('Ramesh ', '9901733369', 'rameshas@anantatech.com', 'Yelekhan'),
('Swarna', '9632595904', 'rameshas2105@gmail.com', 'Vidyaranyaapura '),
('Ramesh', '9901733369', 'rameshas2105@gmail.com', '47A 1st, Cross ,SAINAGAR 2ND STAGE, 2nd Main Rd, Vidyaranyaapura Post, Chikkabettahalli, Bengaluru, Karnataka'),
('test', '9901733369', 'rameshas2105@gmail.com', '47A 1st, Cross ,SAINAGAR 2ND STAGE, 2nd Main Rd, Vidyaranyaapura Post, Chikkabettahalli, Bengaluru, Karnataka'),
('ramesh', '9901733369', 'rameshas2105@gmail.com', '47A 1st, Cross ,SAINAGAR 2ND STAGE, 2nd Main Rd, Vidyaranyaapura Post, Chikkabettahalli, Bengaluru, Karnataka'),
('test', 'test', 'rameshas2105@gmail.com', '47A 1st, Cross ,SAINAGAR 2ND STAGE, 2nd Main Rd, Vidyaranyaapura Post, Chikkabettahalli, Bengaluru, Karnataka 560097'),
('Test', 'Test', 'rameshas2105@gmail.com', '47A 1st, Cross ,SAINAGAR 2ND STAGE, 2nd Main Rd, Vidyaranyaapura Post, Chikkabettahalli, Bengaluru, Karnataka 560097'),
('RAMESH', '9901733369', 'rameshas2105@gmail.com', '47A 1st, Cross ,SAINAGAR 2ND STAGE, 2nd Main Rd, Vidyaranyaapura Post, Chikkabettahalli, Bengaluru, Karnataka'),
('Ramesh', '9901733369', 'rameshas2105@gmail.com', '47A 1st, Cross ,SAINAGAR 2ND STAGE, 2nd Main Rd, Vidyaranyaapura Post, Chikkabettahalli, Bengaluru, Karnataka'),
('Guru Raghavendran S ', '9535633007', 'sguragha@gmail.com', 'Vidyaranyaapura '),
('Kanishk', '7019863826', 'kanishk.cse@skit.org.in', '001 viswas paradigam Ms palya vidyranayapura bangalore'),
('Sagar', '9591451419', 'Sagar.venkatesh1989@gmail.com', 'Vidyaranyaapura Bangalore '),
('YantraByte Solutions ', '9901733369', 'yantrabyte.solutions@gmail.com', 'Vidyaranyaapura '),
('Riyaz', '8147773763', 'riyaz.shaikk@gmail.com', 'Singapura,M.s.palya, Bangalore '),
('N Prashanth ', '9164154505', 'prashanthraju2004@gmail.com', '#417/7 , G2 , 4th cross , A sector , Yelahanka - Newtown'),
('Aarmour Surveillance', '9902583833', 'info@aarmour.asia', 'Sheshadripuram'),
('Aarmour Surveillance ', '9902583833', 'info@aarmour.asia', 'Sheshadripuram'),
('Aarmour Surveillance ', '9902583833', 'info@aarmour.asia', 'Sheshadripuram') ON CONFLICT DO NOTHING;
```

Migration file: 20260516150000_migrate_service_tickets.sql

```
INSERT INTO "service_tickets" (ticket_number, customer_name, customer_phone, customer_email, device_type, issue_description, status) VALUES
('YBS-OLD-40006', 'Manjesh', '8660701899', 'tmanjesh.7712@gmail.com', 'Hp', 'Software installation', 'in-progress'),
('YBS-OLD-92677', 'Dr Sreelekshmi', '9207199609', 'vidyaranayapuraclinicreception@gmail.com', 'CPU', 'Not working - ', 'open'),
('YBS-OLD-21878', 'Girish Malige', '9886253098', 'girimalige@yahoo.co.in', 'HP Laptop', 'Not booting', 'open'),
('YBS-OLD-80302', 'Mallarswami nonvinkere', '9731846846', 'maliar@gmail.com', 'Lenovo ideapad laptop', 'Keyboard', 'open'),
('YBS-OLD-85705', 'Sagar', '9591451419', 'sagar.venkatesh1989@gmail.com', 'Dell', 'Not working', 'open'),
('YBS-OLD-34063', 'Girish', '9945408547', 'giri1119@gmail.com', 'CPU', 'CPU Not starting. Some burning smell detected after UPS switched on.', 'open'),
('YBS-OLD-79312', 'Pawan', '9901733369', 'YantraByte.solutions@gmail.com', 'Dell laptop with charger', 'No display', 'open'),
('YBS-OLD-36258', 'Iibs', '9901733369', 'rameshas2105@gmail.com', 'Dell laptop', 'Bios password need to be removed', 'open'),
('YBS-OLD-28847', 'Akhil Rajput', '8548035733', 'Andajay200@gmail.com', 'Printer', 'Printer head issue, and service', 'open'),
('YBS-OLD-89123', 'yashwant', '9901733369', 'rameshas2105@gmail.com', 'Monitor', 'not working', 'open'),
('YBS-2026-2027-001', 'test', '9999999999', 'rameshas2105@gmail.com', 'monitor', 'not working', 'open'),
('YBS-2026-2027-002', 'test', '9999999999', 'rameshas2105@gmail.com', 'printer', 'not working', 'open'),
('YBS-2026-2027-003', 'test', '9999999999', 'sysadmin@iibsonline.com', 'printer', 'not working', 'open'),
('YBS-OLD-97026', 'Ramesh', '9901733369', 'rameshas@anantatech.com', 'OPEN', 'https://drive.google.com/file/d/1oZJW8wLIvSuF5e7iT4yX5tJ37u2RB5nB/view?usp=drive', 'open'),
('YBS-OLD-76973', 'Swarna', '9632595904', 'rameshas2105@gmail.com', 'OPEN', 'https://drive.google.com/file/d/1iKiv0B2Xhy45vQqa6ibw2j5dAuQ0tJxo/view?usp=drive', 'open'),
('YBS-2027', 'Ramesh', '9901733369', 'rameshas2105@gmail.com', 'Laptop', 'NOT booting', 'open'),
('YBS-120', 'test', '9901733369', 'rameshas2105@gmail.com', 'Laptop', 'not booting', 'open'),
('YBS-120', 'ramesh', '9901733369', 'rameshas2105@gmail.com', 'Laptop', 'not working', 'open'),
('YBS-121', 'test', 'test', 'rameshas2105@gmail.com', 'CPU', 'not working', 'open'),
('YBS-122', 'Test', 'Test', 'rameshas2105@gmail.com', 'Desktop', 'not working', 'open'),
('YBS-123', 'RAMESH', '9901733369', 'rameshas2105@gmail.com', 'Desktop', 'not booting', 'closed'),
('YBS-OLD-11092', 'Ramesh', '9901733369', 'rameshas2105@gmail.com', 'Printer', 'Not printing', 'open'),
('YBS-OLD-98412', 'Guru Raghavendran S', '9535633007', 'sguragha@gmail.com', 'HP', 'Windows not working', 'open'),
('YBS-OLD-17922', 'Kanishk', '7019863826', 'kanishk.cse@skit.org.in', 'Cpu', 'Can''t boot the system', 'open'),
('YBS-OLD-89834', 'Sagar', '9591451419', 'Sagar.venkatesh1989@gmail.com', 'Lenovo laptop', 'System slow, bottom panel not showing', 'open'),
('YBS-OLD-53397', 'YantraByte Solutions', '9901733369', 'yantrabyte.solutions@gmail.com', 'Printer', 'Not working', 'open'),
('YBS-OLD-62846', 'Riyaz', '8147773763', 'riyaz.shaikk@gmail.com', 'Lenovo laptop', 'Sudden shutdown', 'open'),
('YBS-OLD-92851', 'N Prashanth', '9164154505', 'prashanthraju2004@gmail.com', 'Dell Laptop', 'Laptop for service', 'open'),
('YBS-TKT-140526-035', 'Aarmour Surveillance', '9902583833', 'info@aarmour.asia', 'Hp Laptop', 'Laptop charging pin damaged and laptop hanging issue', 'in-progress'),
('YBS-TKT-140526-033', 'Aarmour Surveillance', '9902583833', 'info@aarmour.asia', 'DELL laptop', 'Not powering on', 'in-progress'),
('YBS-TKT-140526-034', 'Aarmour Surveillance', '9902583833', 'info@aarmour.asia', 'Hp Laptop', 'Laptop charging pin damaged and laptop hanging issue', 'in-progress')
```


Migration file: 20260520120000_add_purchases_and_suppliers.sql

```
-- Create suppliers table for the Billing Software
CREATE TABLE IF NOT EXISTS public.suppliers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,
    phone TEXT,
    email TEXT,
    address TEXT,
    gstin TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Create purchases table for the Billing Software
CREATE TABLE IF NOT EXISTS public.purchases (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    purchase_no TEXT UNIQUE NOT NULL,
    supplier_id UUID REFERENCES public.suppliers(id) ON DELETE SET NULL,
    supplier_name TEXT NOT NULL,
    date TEXT NOT NULL,
    items JSONB NOT NULL DEFAULT '[]'::jsonb,
    subtotal NUMERIC NOT NULL DEFAULT 0,
    discount NUMERIC NOT NULL DEFAULT 0,
    tax NUMERIC NOT NULL DEFAULT 0,
    round_off NUMERIC NOT NULL DEFAULT 0,
    grand_total NUMERIC NOT NULL DEFAULT 0,
    amount_paid NUMERIC NOT NULL DEFAULT 0,
    balance_due NUMERIC NOT NULL DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Enable RLS Policies
ALTER TABLE public.suppliers ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.purchases ENABLE ROW LEVEL SECURITY;

-- RLS policies for suppliers table
CREATE POLICY "Allow read access for authenticated users on suppliers"
    ON public.suppliers FOR SELECT
    USING (auth.role() = 'authenticated');

CREATE POLICY "Allow insert access for authenticated users on suppliers"
    ON public.suppliers FOR INSERT
    WITH CHECK (auth.role() = 'authenticated');

CREATE POLICY "Allow update access for authenticated users on suppliers"
    ON public.suppliers FOR UPDATE
    USING (auth.role() = 'authenticated');

CREATE POLICY "Allow delete access for authenticated users on suppliers"
    ON public.suppliers FOR DELETE
    USING (auth.role() = 'authenticated');

-- RLS policies for purchases table
CREATE POLICY "Allow read access for authenticated users on purchases"
    ON public.purchases FOR SELECT
    USING (auth.role() = 'authenticated');

CREATE POLICY "Allow insert access for authenticated users on purchases"
    ON public.purchases FOR INSERT
    WITH CHECK (auth.role() = 'authenticated');

CREATE POLICY "Allow update access for authenticated users on purchases"
    ON public.purchases FOR UPDATE
    USING (auth.role() = 'authenticated');

CREATE POLICY "Allow delete access for authenticated users on purchases"
    ON public.purchases FOR DELETE
    USING (auth.role() = 'authenticated');
```

Migration file: 20260522150000_add_customers_and_invoice_payment_fields.sql

```
-- Add customer master records and payment tracking to invoices.
CREATE TABLE IF NOT EXISTS public.customers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,
    phone TEXT UNIQUE,
    email TEXT,
    address TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

ALTER TABLE public.customers ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Allow read access for authenticated users on customers"
    ON public.customers FOR SELECT
    USING (auth.role() = 'authenticated');

CREATE POLICY "Allow insert access for authenticated users on customers"
    ON public.customers FOR INSERT
    WITH CHECK (auth.role() = 'authenticated');

CREATE POLICY "Allow update access for authenticated users on customers"
    ON public.customers FOR UPDATE
    USING (auth.role() = 'authenticated');

CREATE POLICY "Allow delete access for authenticated users on customers"
    ON public.customers FOR DELETE
    USING (auth.role() = 'authenticated');

ALTER TABLE public.invoices
    ADD COLUMN IF NOT EXISTS customer_id UUID REFERENCES public.customers(id) ON DELETE SET NULL,
    ADD COLUMN IF NOT EXISTS payment_mode TEXT NOT NULL DEFAULT 'Not specified',
    ADD COLUMN IF NOT EXISTS payment_status TEXT NOT NULL DEFAULT 'Due',
    ADD COLUMN IF NOT EXISTS due_date TEXT;

-- Populate customers from invoices
INSERT INTO public.customers (name, phone, email, address)
SELECT DISTINCT ON (COALESCE(NULLIF(BTRIM(phone), ''), LOWER(BTRIM(customer_name))))
    customer_name,
    NULLIF(BTRIM(phone), ''),
    NULLIF(BTRIM(email), ''),
    NULLIF(BTRIM(address), '')
FROM public.invoices
WHERE NULLIF(BTRIM(customer_name), '') IS NOT NULL
ORDER BY COALESCE(NULLIF(BTRIM(phone), ''), LOWER(BTRIM(customer_name))), created_at DESC
ON CONFLICT (phone) DO NOTHING;

UPDATE public.invoices inv
SET customer_id = cust.id
FROM public.customers cust
WHERE inv.customer_id IS NULL
    AND NULLIF(BTRIM(inv.phone), '') IS NOT NULL
    AND cust.phone = NULLIF(BTRIM(inv.phone), '');

UPDATE public.invoices inv
SET customer_id = cust.id
FROM public.customers cust
WHERE inv.customer_id IS NULL
    AND LOWER(BTRIM(inv.customer_name)) = LOWER(BTRIM(cust.name));

UPDATE public.invoices
SET payment_status = CASE
    WHEN doc_type = 'Quotation' THEN 'Estimate'
    WHEN COALESCE(balance_due, 0) <= 0 THEN 'Paid'
    WHEN COALESCE(advance_paid, 0) > 0 THEN 'Partial'
    ELSE 'Due'
END
WHERE payment_status IS NULL OR payment_status = 'Due';
```

Migration file: 20260522170000_start_fresh_customer_master.sql

```
-- Start a fresh Customer Master without deleting old invoices, quotations, or purchases.
DO $$
BEGIN
    IF to_regclass('public.invoices') IS NOT NULL
       AND to_regclass('public.customers') IS NOT NULL THEN
        UPDATE public.invoices
        SET customer_id = NULL
        WHERE customer_id IS NOT NULL;

        DELETE FROM public.customers;
    END IF;
END $$;

INSERT INTO public.site_settings (key, value)
VALUES ('billing_customer_master_fresh_started_at', '2026-05-22T19:00:00+05:30')
ON CONFLICT (key) DO UPDATE
SET value = EXCLUDED.value,
    updated_at = NOW();

INSERT INTO public.site_settings (key, value)
VALUES ('billing_documents_fresh_started_at', NOW()::TEXT)
ON CONFLICT (key) DO NOTHING;
```

Migration file: 20260522200000_public_service_request_form.sql

```
ALTER TABLE public.service_tickets
  ADD COLUMN IF NOT EXISTS customer_address text;

DROP POLICY IF EXISTS "Public can create service tickets" ON public.service_tickets;

CREATE POLICY "Public can create service tickets"
  ON public.service_tickets
  FOR INSERT
  TO anon
  WITH CHECK (
    ticket_number LIKE 'YBS-service-Ticket %'
    AND NULLIF(BTRIM(customer_name), '') IS NOT NULL
    AND NULLIF(BTRIM(customer_phone), '') IS NOT NULL
    AND NULLIF(BTRIM(issue_description), '') IS NOT NULL
    AND status = 'open'
    AND priority IN ('low', 'medium', 'high', 'urgent')
    AND assigned_to IS NULL
    AND notes IS NULL
  );
```

Migration file: 20260523000000_ticket_sequence.sql

```
CREATE OR REPLACE FUNCTION get_next_service_ticket_number()
RETURNS text
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
    next_suffix integer := 1;
    last_ticket text;
    month_start date;
    month_end date;
BEGIN
    month_start := date_trunc('month', current_date)::date;
    month_end := (date_trunc('month', current_date) + interval '1 month - 1 day')::date;

    SELECT ticket_number INTO last_ticket
    FROM public.service_tickets
    WHERE created_at >= month_start AND created_at <= month_end
    ORDER BY created_at DESC
    LIMIT 1;

    IF last_ticket IS NOT NULL THEN
        next_suffix := (regexp_replace(last_ticket, '^\.*-(\d+)$', '\1'))::integer + 1;
    END IF;

    RETURN 'YBS-service-Ticket ' || to_char(current_date, 'DDMMYYYY') || '-' || lpad(next_suffix::text, 3, '0');
END;
$$;
```

Migration file: 20260527130000_add_payments_column.sql

```
ALTER TABLE public.invoices ADD COLUMN IF NOT EXISTS payments JSONB DEFAULT '[]'::jsonb;
```

Migration file: 20260527150000_create_invoices_storage_bucket.sql

```
-- Create a public storage bucket for invoice PDFs
INSERT INTO storage.buckets (id, name, public)
VALUES ('invoices', 'invoices', true)
ON CONFLICT (id) DO NOTHING;

-- Allow authenticated users to upload PDFs
CREATE POLICY "Authenticated users can upload invoice PDFs"
ON storage.objects FOR INSERT
TO authenticated
WITH CHECK (bucket_id = 'invoices');

-- Allow authenticated users to update/overwrite PDFs
CREATE POLICY "Authenticated users can update invoice PDFs"
ON storage.objects FOR UPDATE
TO authenticated
USING (bucket_id = 'invoices');

-- Allow anyone to read/download invoice PDFs (public access)
CREATE POLICY "Public can read invoice PDFs"
ON storage.objects FOR SELECT
TO public
USING (bucket_id = 'invoices');
```

Migration file: 20260527160000_add_external_repairs.sql

```
-- Create external_repairs table to track materials sent out for external service
CREATE TABLE IF NOT EXISTS public.external_repairs (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  ticket_number text REFERENCES public.service_tickets(ticket_number) ON DELETE SET NULL,
  material_name text NOT NULL,
  serial_number text,
  sent_to text NOT NULL,
  sent_date date NOT NULL DEFAULT CURRENT_DATE,
  expected_return_date date,
  status text NOT NULL DEFAULT 'sent', -- 'sent', 'received', 'cancelled'
  received_date date,
  cost numeric DEFAULT 0,
  notes text,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- Enable RLS
ALTER TABLE public.external_repairs ENABLE ROW LEVEL SECURITY;

-- Create policy for authenticated users (admins) to have full access
CREATE POLICY "Admins have full access to external_repairs"
ON public.external_repairs
FOR ALL
TO authenticated
USING (true)
WITH CHECK (true);
```


Migration file: 20260527170000_erp_extensions.sql

```
-- Create serialized_items table to track individual product serials
CREATE TABLE IF NOT EXISTS public.serialized_items (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id uuid REFERENCES public.products(id) ON DELETE CASCADE,
  serial_number text UNIQUE NOT NULL,
  status text NOT NULL DEFAULT 'in_stock', -- 'in_stock', 'sold', 'returned_to_vendor', 'written_off'
  purchase_id uuid REFERENCES public.purchases(id) ON DELETE SET NULL,
  invoice_no text, -- text field referencing invoice_no
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- Create expenses table to track operational costs
CREATE TABLE IF NOT EXISTS public.expenses (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  category text NOT NULL, -- 'Rent', 'Salaries', 'Utilities', 'Tools', 'Marketing', 'Other'
  amount numeric NOT NULL DEFAULT 0,
  date date NOT NULL DEFAULT CURRENT_DATE,
  description text,
  created_at timestamptz DEFAULT now()
);

-- Create amc_contracts table to track yearly AMC packages
CREATE TABLE IF NOT EXISTS public.amc_contracts (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  client_name text NOT NULL,
  client_email text,
  client_phone text NOT NULL,
  contract_value numeric NOT NULL DEFAULT 0,
  start_date date NOT NULL,
  end_date date NOT NULL,
  status text NOT NULL DEFAULT 'active', -- 'active', 'expired', 'renewed'
  notes text,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- Enable RLS Policies on new tables
ALTER TABLE public.serialized_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.expenses ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.amc_contracts ENABLE ROW LEVEL SECURITY;

-- Admins full access policies
CREATE POLICY "Admins have full access to serialized_items"
  ON public.serialized_items FOR ALL TO authenticated USING (true) WITH CHECK (true);

CREATE POLICY "Admins have full access to expenses"
  ON public.expenses FOR ALL TO authenticated USING (true) WITH CHECK (true);

CREATE POLICY "Admins have full access to amc_contracts"
  ON public.amc_contracts FOR ALL TO authenticated USING (true) WITH CHECK (true);
```

Migration file: 20260527180000_add_supplier_invoice_no_to_purchases.sql

```
-- Add supplier_invoice_no column to public.purchases table if not exists
ALTER TABLE public.purchases ADD COLUMN IF NOT EXISTS supplier_invoice_no text;
```

Migration file: 20260529100000_ensure_public_schema_grants.sql

```
-- Grant access to all existing tables and sequences in the public schema
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO anon, authenticated, service_role;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO anon, authenticated, service_role;

-- Ensure future tables and sequences automatically get these privileges when created by the postgres role
ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO anon, authenticated, service_role;

ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA public
    GRANT USAGE, SELECT ON SEQUENCES TO anon, authenticated, service_role;

-- Ensure future tables and sequences automatically get these privileges when created by the current role
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO anon, authenticated, service_role;

ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT USAGE, SELECT ON SEQUENCES TO anon, authenticated, service_role;
```

Migration file: 20260529_add_ticket_quote_fields.sql

```
-- Migration: Add quote approval and diagnostic fields to service_tickets
-- Run this in your Supabase SQL Editor

ALTER TABLE public.service_tickets
  ADD COLUMN IF NOT EXISTS estimated_cost NUMERIC DEFAULT NULL,
  ADD COLUMN IF NOT EXISTS diagnostic_image_url TEXT DEFAULT NULL,
  ADD COLUMN IF NOT EXISTS client_approval_status TEXT DEFAULT NULL;

-- Add customer_address column if not present (used by ServiceRequest form)
ALTER TABLE public.service_tickets
  ADD COLUMN IF NOT EXISTS customer_address TEXT DEFAULT NULL;

-- Add technician_notes column if not present (used by TrackTicket)
ALTER TABLE public.service_tickets
  ADD COLUMN IF NOT EXISTS technician_notes TEXT DEFAULT NULL;

-- Allow anonymous users to read limited ticket data for tracking page
-- (They already need ticket_number + phone to look up, so this is safe)
CREATE POLICY IF NOT EXISTS "Public can read own tickets by number and phone"
  ON public.service_tickets FOR SELECT
  TO anon
  USING (true);

-- Allow anonymous users to update client_approval_status only
CREATE POLICY IF NOT EXISTS "Public can update approval status"
  ON public.service_tickets FOR UPDATE
  TO anon
  USING (true)
  WITH CHECK (true);

-- Note: The RLS policies above are permissive because the app-level code
-- already requires ticket_number + phone match to find a ticket.
-- For production hardening, consider a Supabase Edge Function instead.
```